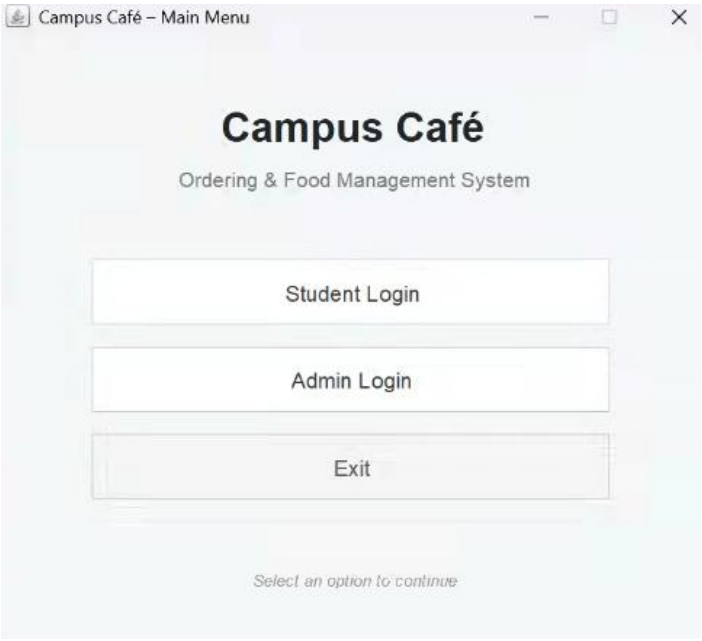
# OBJECT-ORIENTED PROGRAMMING ASSIGNMENT

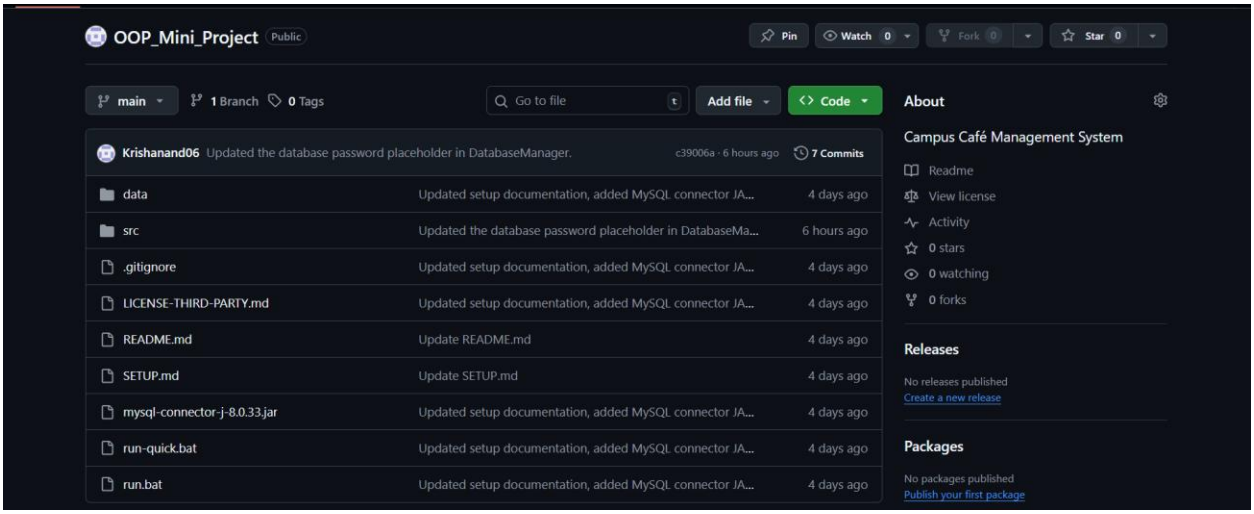
# (CS F213)

# Semester 1 2025- 2026

Campus Cafe Ordering & Food Management System




https://github.com/Krishanand06/OOP_Mini_Project



## Team Members:

| |
|---|
| Prashanth Ananthakrishnan – 2024A7PS0329U |
| Krish Anand – 2024A7PS0282U |
| Sanjit Madankumar – 2024A7PS0275U |
| Pranav A – 2024A7PS0285U |
| Farhan Kazi – 2024A7PS0126U |

# Table of Contents


| Topic | Page Number |
|---|---|
| Abstract | 4 |
| Introduction | 5 |
| System Analysis and Design | 7 |
| Use Case Diagram | 10 |
| Flow Charts | 11 |
| Class Diagram | 13 |
| Major Modules | 32 |
| Implementation Analysis | 34 |
| Conclusion & Future Work | 42 |
| References: | 44 |

# Abstract

Campus Café is a food ordering and management system aimed at simplifying all food-related activities within college campuses, both for students and administrators. The system digitizes common activities, including canteen ordering, wallet payments, mess plan subscriptions, event-catering requests, and student feedback, through a user-friendly Java Swing interface. It allows students to view menu items, manage their Campus Wallet, print receipts, provide feedback, and create catering requests for club events. Hostellers have an extra option on their Student Menu, whereby they can choose their Mess Plans. For an administrator, the system provides a single panel to view canteen orders, act on catering requests, review feedback entries, track active mess subscriptions, or examine trending food choices. The project has been developed using OOP concepts like inheritance, file handling, interfaces, and exception handling in combination with file-based data storage to provide a seamless food-service experience at campus level.

# Introduction

## 1. Background and Motivation:

Catering-related activities, such as canteen ordering, mess plan management, event catering, and feedback collection, are performed through separate, disorganized processes on most university campuses. Students often waste time, energy and resources trying to understand these different systems and do not have one place where they can manage their daily food and catering needs with ease. Administratively, staff members face difficulties in keeping track of the catering requests, monitoring food trends, and effectively managing the routine operations.

These issues point to the need for a unified digital solution that brings all food service and catering related functions under one integrated system. The idea of the Campus Café project was born from recognizing these practical challenges within universities. A streamlined platform which combines ordering, payments, feedback, and administration would not only provide better convenience for students but also enhance transparency and improve efficiency for administrators. The motivation behind this project is to develop such a system that simplifies campus food-service and catering workflows while offering a more simplified, organized and user-friendly experience for both Students and Administrators who are on Campus.

## 2. Objectives:

The key objectives of this project are:

- To create an integrated platform for food ordering in College Canteens, event catering in Campuses, Hostel mess plan subscriptions, wallet payments, and feedback management.
- To provide students with a user-friendly interface through which they can easily order, make secure transactions, and access other food-related services.
- To provide administrators with tools to track orders, respond to catering requests, review feedback, and analyze food popularity thus giving administrators clarity while planning day to day operations.
- To implement the core principles of OOP, namely inheritance, polymorphism, interfaces, and exception handling, into a software solution.
- To ensure reliable file-based data persistence so that user accounts, transactions, feedback, and system records are safely stored.

## 3. Scope:

The scope of the system includes:

- Student Module:

The services provided within the portal include registration and login, canteen ordering, Campus Wallet top-up, receipt generation, event catering requests, mess plan subscription for hostellers, and feedback submission.

- Admin Module:

View and update status of canteen orders(pending / approved), manage catering requests (approve/reject), view feedback given by Students, view active mess plans, and trending menu items.

**Technical Scope:**

- ✓ Implementation using Java Swing for GUI
- ✓ Use of OOP concepts throughout the design
- ✓ File-based storage for users, orders, catering requests, feedback, receipts and mess plans
- ✓ Modular classes for students, admins, wallet, orders, menus, mess plans, catering items

# System Analysis and Design:

**Requirement Analysis:**

Functional Requirements:

- **User Registration:** The system shall allow a new student to register by providing roll number, name, email, password and student type (Hosteller / Day Scholar).
- **Student Login:** The system shall allow a registered student to log in using roll number and password.
- **Admin Login:** The system shall allow an administrator to log in using a valid admin username and password.
- **View Menu:** The system shall display the canteen menu with item name, category and price.
- **Select Items and Quantity:** The system shall allow a student to select one or more menu items and specify the quantity for each.
- **Manage Cart:** The system shall allow the student to add items to a cart, view current order items, clear the cart and see the running total.
- **Place Order:** The system shall allow the student to place an order and pay using the Campus Wallet if sufficient balance is available.
- **Generate Receipt:** After a successful payment, the system shall generate a receipt file containing order details, student details, date/time and total amount.
- **View Wallet Balance:** The system shall display the current wallet balance of the logged-in student.
- **Top-up Wallet:** The system shall allow the student to top-up the wallet by entering an amount.
- **Record Transactions:** The system shall record each top-up and payment as a transaction entry with date/time, type (CREDIT/DEBIT), amount and description.
- **View Transaction History**: The system shall allow the student to view their wallet transaction history.
- **View Available Mess Plans:** The system shall display available mess plans (Basic, Premium, Weekend) with price, duration and description.
- **Subscribe to Mess Plan:** The system shall allow a hosteller to subscribe to one mess plan at a time.
- **Store Mess Subscription:** The system shall store the mess plan subscription with student details, plan name, start date, end date and status.
- **Submit Catering Request:** The system shall allow a student to submit a catering request by entering event name, date, time, venue, expected attendees and selecting catering items with quantities.

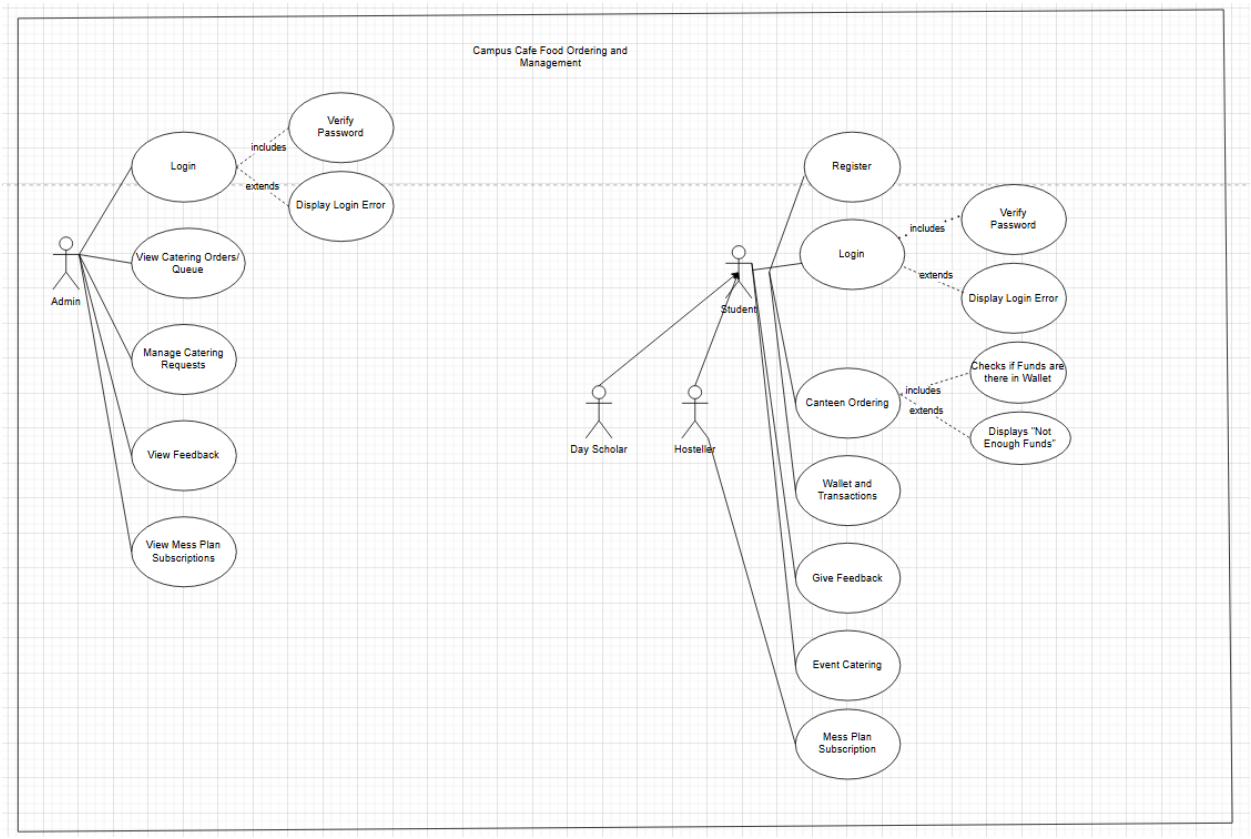
- **Store Catering Request:** The system shall store each catering request with a unique request ID, event details, requested items and status (PENDING/APPROVED/REJECTED).
- **Submit Feedback:** The system shall allow a student to submit feedback by selecting a category (Canteen, Mess, Event Catering), giving a rating (1–5) and entering comments.
- **View Own Feedback:** The system shall allow a student to view the feedback they have submitted.
- **View and Update Canteen Orders:** The system shall allow the admin to view all canteen orders, see their details and update the status between PENDING and COMPLETED.
- **Manage Catering Requests:** The system shall allow the admin to view all catering requests and change their status to APPROVED or REJECTED.
- **View Feedback Records:** The system shall allow the admin to view all feedback entries with student details, category, rating and comments.
- **View Mess Plan Subscriptions:** The system shall allow the admin to view active mess plan subscriptions for all hosteller students.
- **Show Trending Items:** The system shall calculate and display the top-ordered menu items based on past orders.

Non Functional Requirements:

- **Usability:** The system should provide a clean, intuitive and user-friendly GUI through Java Swing so that students and administrators can easily navigate menus, place orders, and manage operations without prior training.
- **Reliability:** All user actions such as ordering, wallet transactions, feedback submission, and catering requests must be stored accurately using file-based data persistence to avoid data loss.
- **Performance:** The system should load menus, display dashboards, and process operations such as login, order creation, and admin updates quickly, without noticeable delay.
- **Security:** Passwords must be stored securely (not visible in plaintext during login). Admin login must be protected with restricted credentials. Only hostellers should access mess plan features.
- **Maintainability:** The application should use OOP concepts (inheritance, interfaces, polymorphism, exception handling) to ensure the codebase is modular, well-organized and easy to modify or extend in the future.

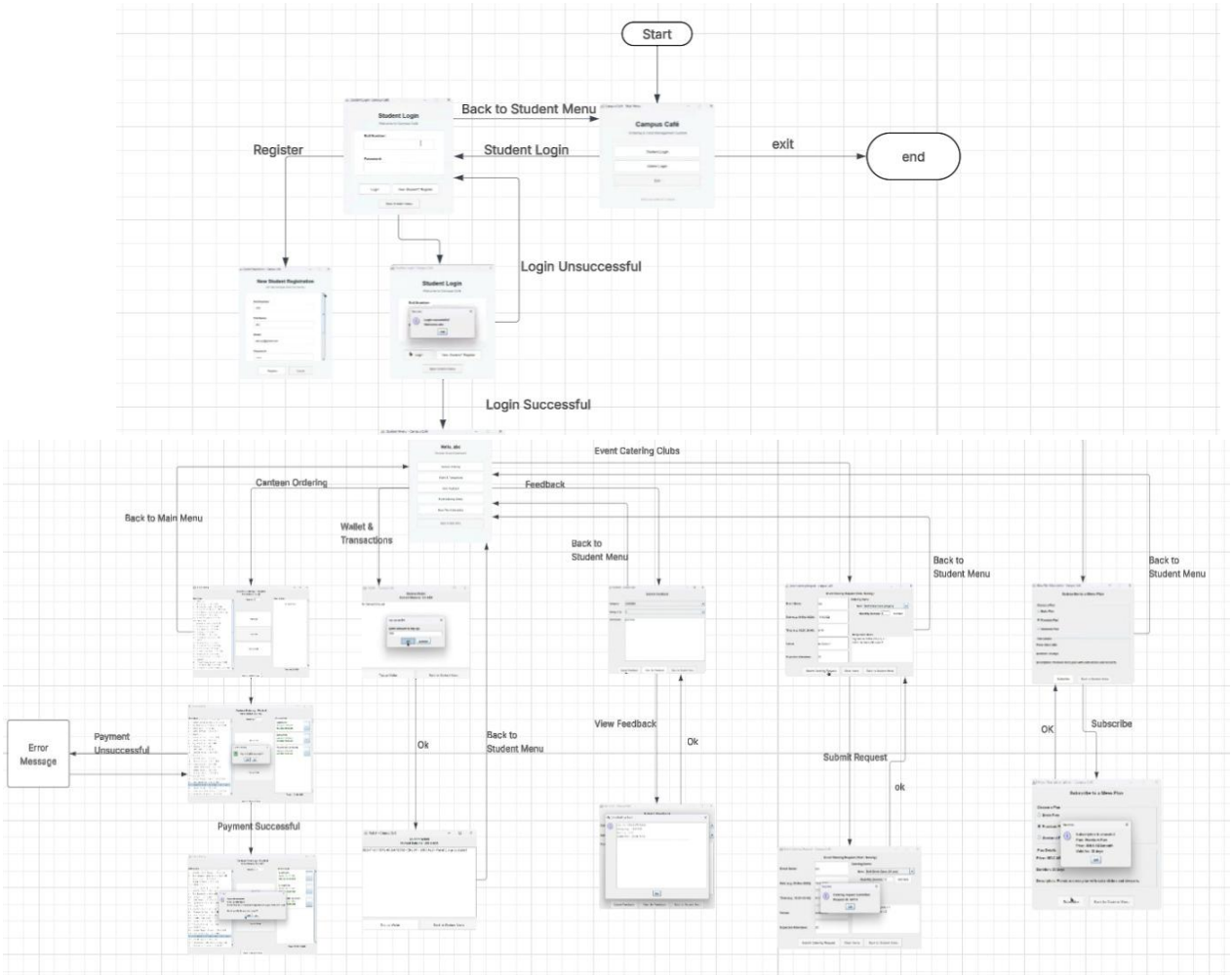
- **Scalability:** The system should be capable of handling an increasing number of students, orders, feedback entries and catering requests without affecting performance.
- **Portability:** As the application is built in Java, it should run on any operating system with a compatible Java Runtime Environment (JRE), including Windows and Linux.
- **Availability:** The system should remain operational as long as the program is running, allowing students and admins to perform tasks at any time without downtime.
- **Data Integrity:** The system must ensure that wallet balances, order totals, mess subscriptions and feedback entries are recorded accurately and consistently across all files.
- **Error Handling:** The system should gracefully handle invalid inputs, missing files, incorrect login attempts, insufficient wallet balance, and other unexpected situations using proper exception handling.

# Use Case Diagram:

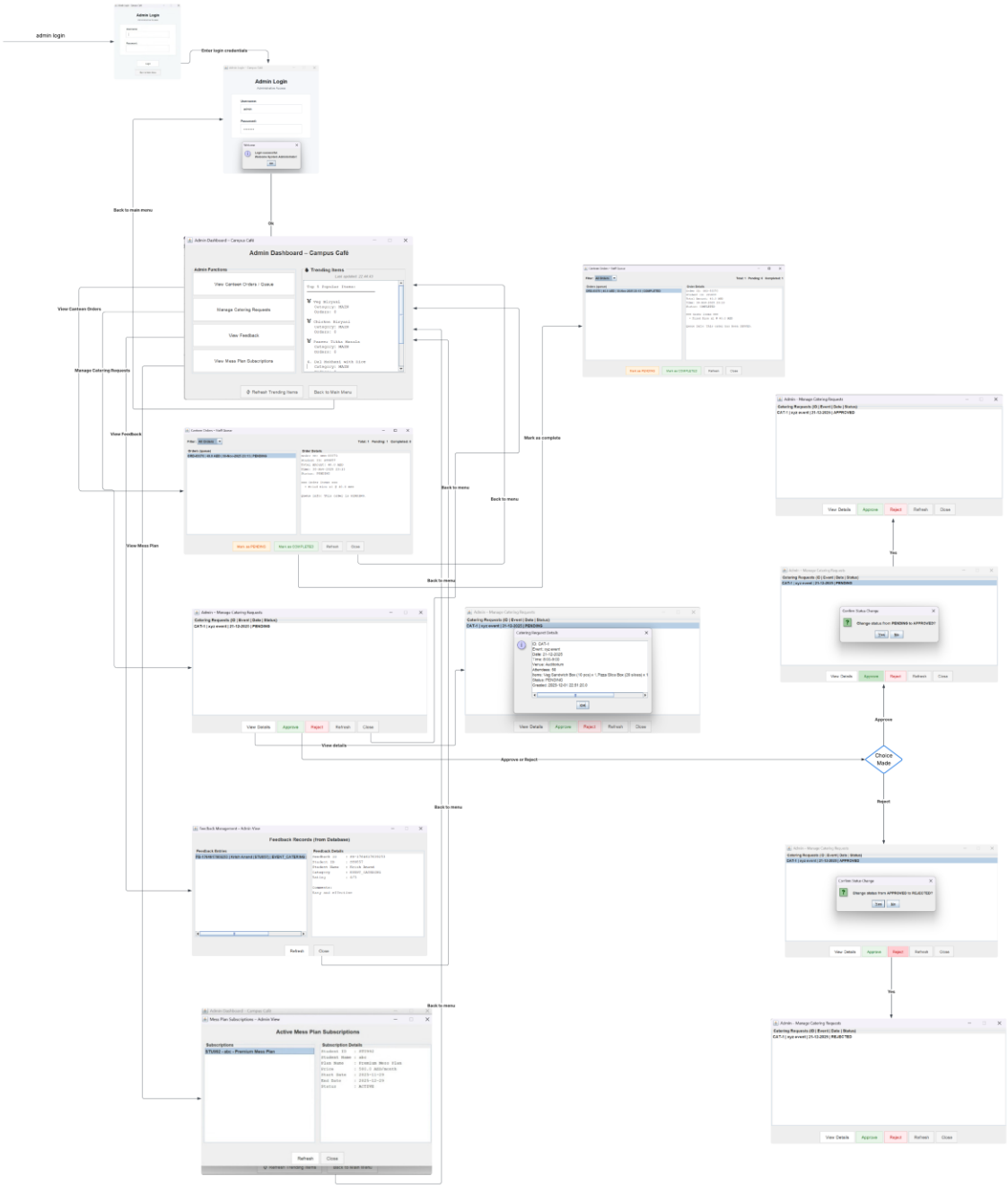


# Flow Charts

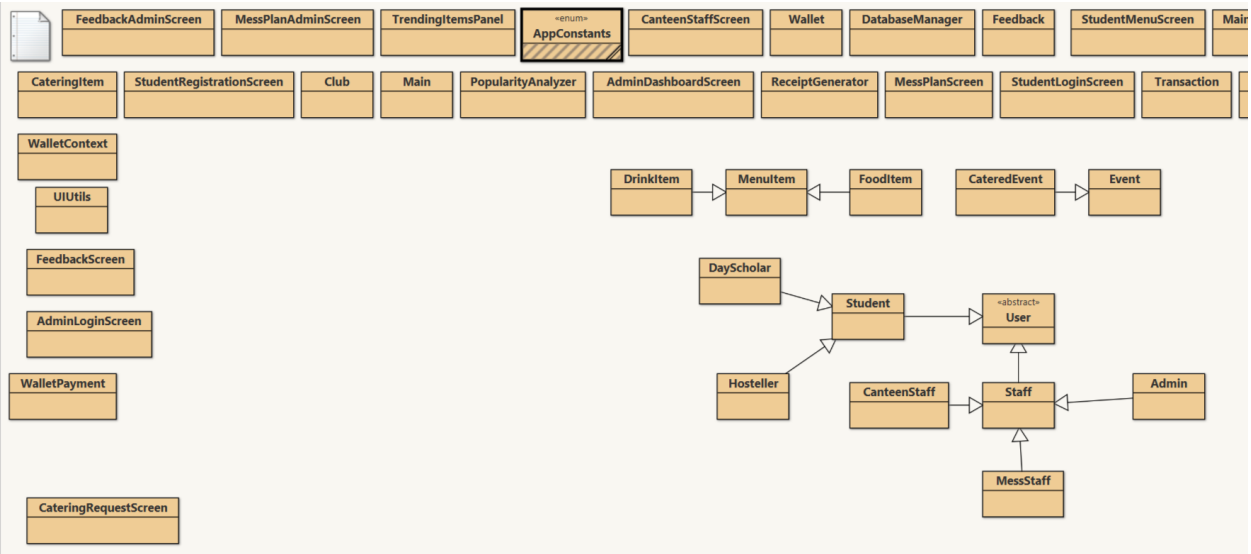
Student login:

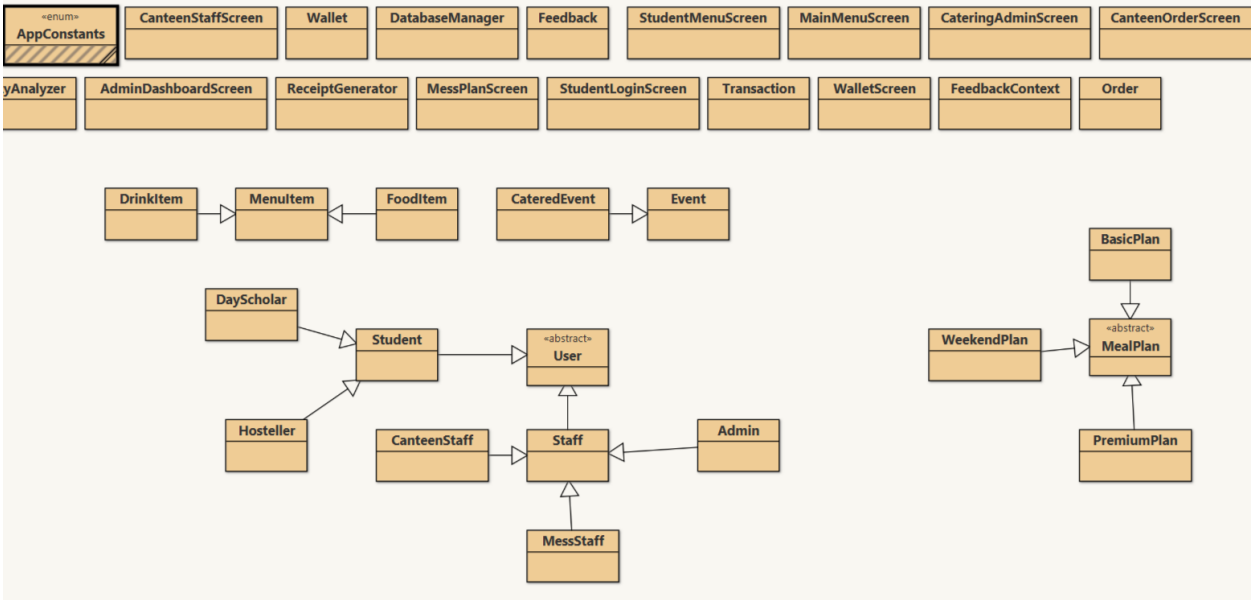


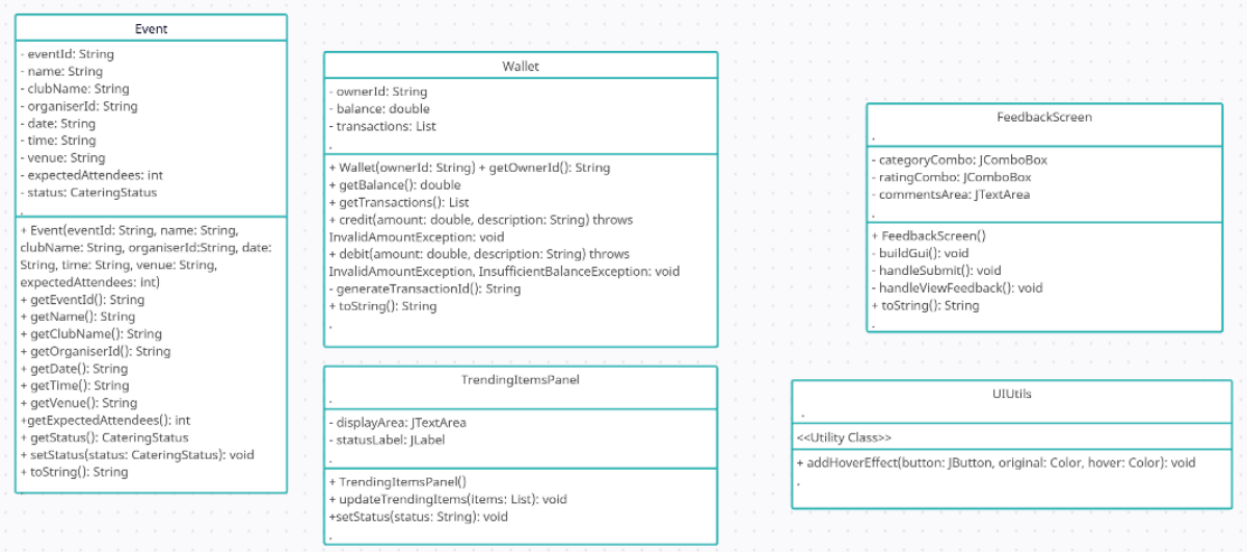
Admin Login:

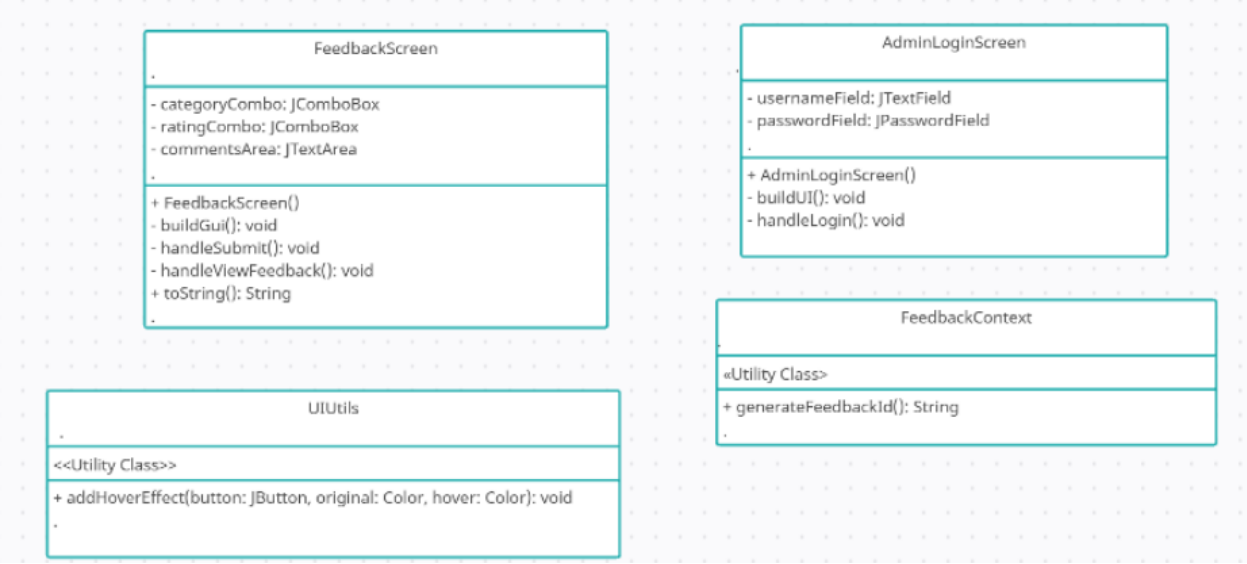

admin login
Enter login credentials
Admin Login
Admin Dashboard – Campus Café
Admin Functions
View Canteen Orders / Queue
Manage Catering Requests
View Feedback
View Mess Plan Subscriptions
Trending Items
Refresh Trending Items
Back to Main Menu
Back to main menu
View Canteen Orders
Manage Catering Requests
View Feedback
View Mess Plan
Mark as complete
Back to menu
Approve or Reject
Choice Made
Approve
Reject
Feedback Records (from Database)
Active Mess Plan Subscriptions
Refresh
Close

# Class Diagram

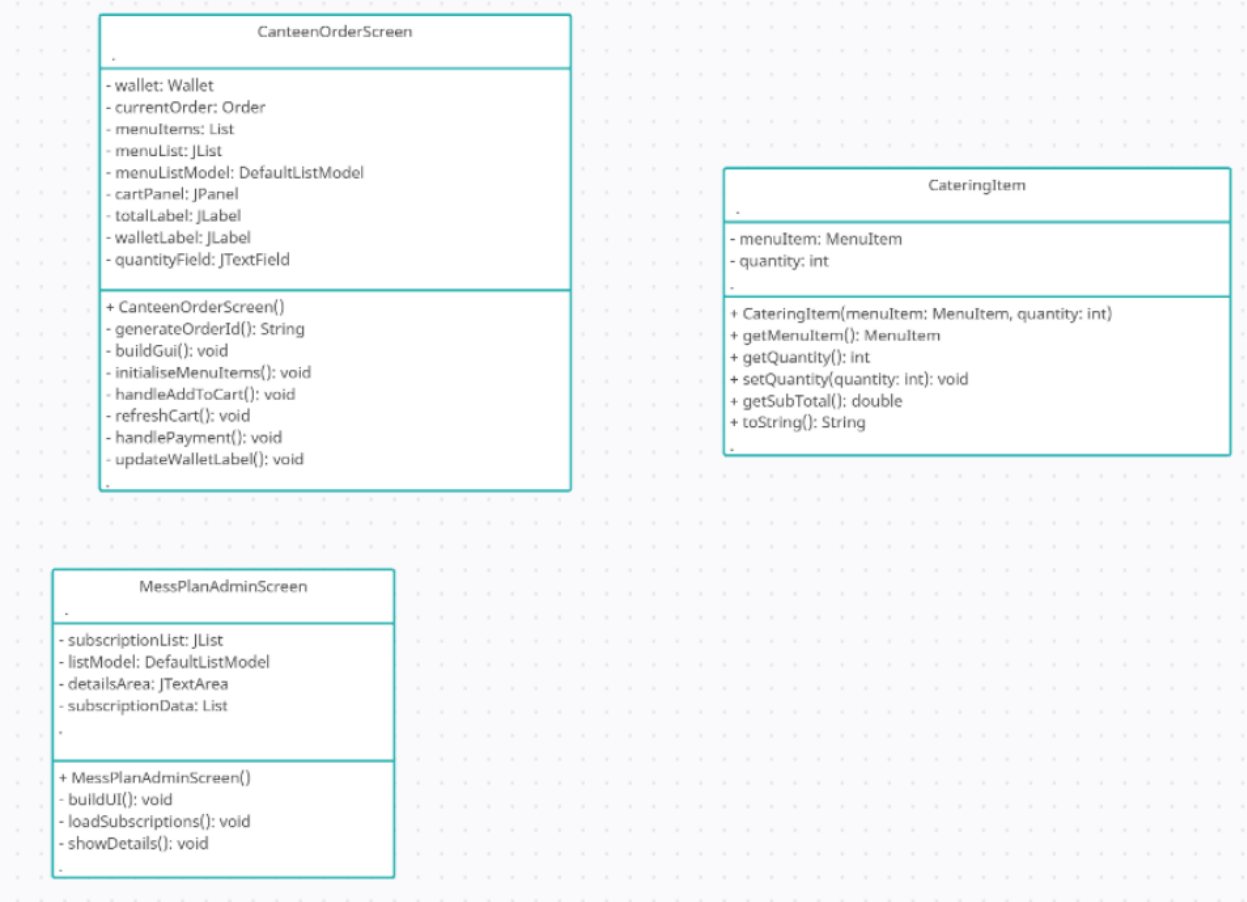


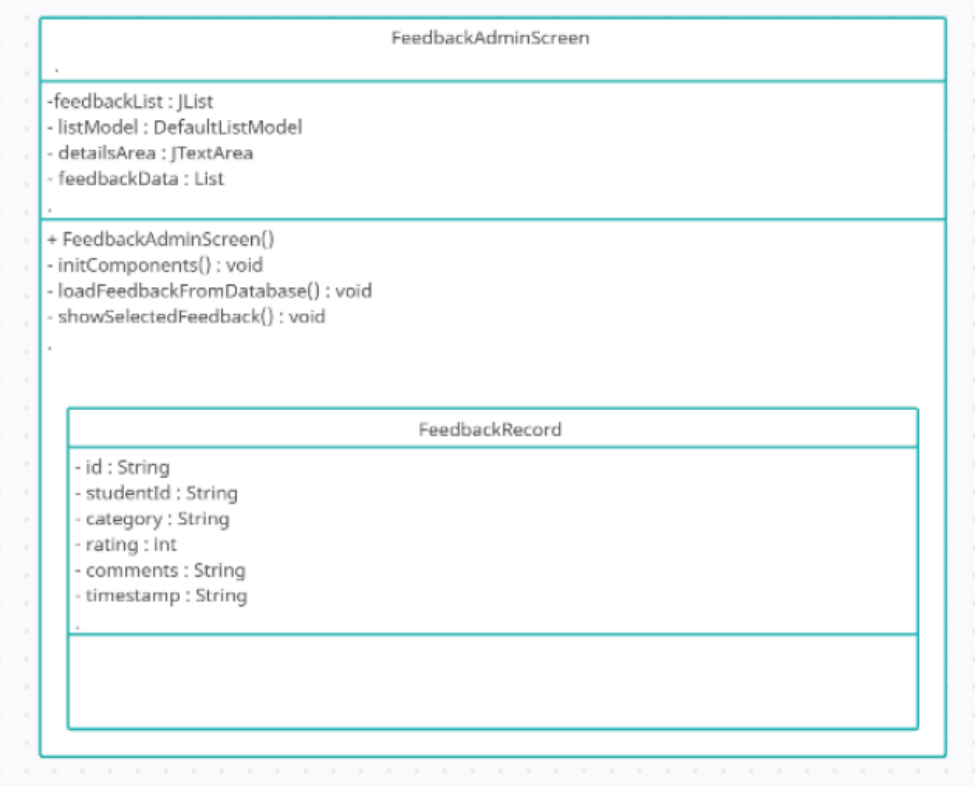



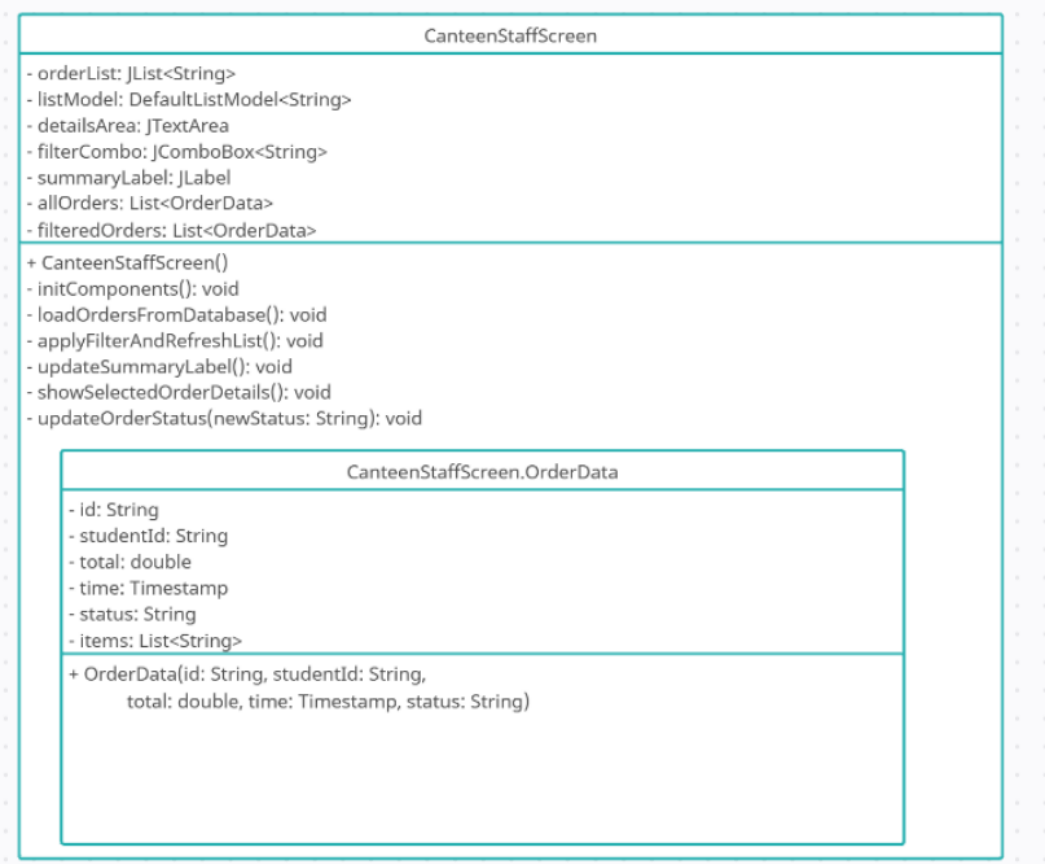

Event
- eventId: String
- name: String
- clubName: String
- organiserId: String
- date: String
- time: String
- venue: String
- expectedAttendees: int
- status: CateringStatus
+ Event(eventId: String, name: String, clubName: String, organiserId:String, date: String, time: String, venue: String, expectedAttendees: int)
+ getEventId(): String
+ getName(): String
+ getClubName(): String
+ getOrganiserId(): String
+ getDate(): String
+ getTime(): String
+ getVenue(): String
+getExpectedAttendees(): int
+ getStatus(): CateringStatus
+ setStatus(status: CateringStatus): void
+ toString(): String
Wallet
- ownerId: String
- balance: double
- transactions: List
+ Wallet(ownerId: String) + getOwnerId(): String
+ getBalance(): double
+ getTransactions(): List
+ credit(amount: double, description: String) throws InvalidAmountException: void
+ debit(amount: double, description: String) throws InvalidAmountException, InsufficientBalanceException: void
- generateTransactionId(): String
+ toString(): String
FeedbackScreen
- categoryCombo: JComboBox
- ratingCombo: JComboBox
- commentsArea: JTextArea
+ FeedbackScreen()
- buildGui(): void
- handleSubmit(): void
- handleViewFeedback(): void
+ toString(): String
TrendingItemsPanel
- displayArea: JTextArea
- statusLabel: JLabel
+ TrendingItemsPanel()
+ updateTrendingItems(items: List): void
+setStatus(status: String): void
UIUtils
<<Utility Class>>
+ addHoverEffect(button: JButton, original: Color, hover: Color): void


FeedbackScreen
- categoryCombo: JComboBox
- ratingCombo: JComboBox
- commentsArea: JTextArea
+ FeedbackScreen()
- buildGui(): void
- handleSubmit(): void
- handleViewFeedback(): void
+ toString(): String
AdminLoginScreen
- usernameField: JTextField
- passwordField: JPasswordField
+ AdminLoginScreen()
- buildUI(): void
- handleLogin(): void
FeedbackContext
«Utility Class»
+ generateFeedbackId(): String
UIUtils
<<Utility Class>>
+ addHoverEffect(button: JButton, original: Color, hover: Color): void

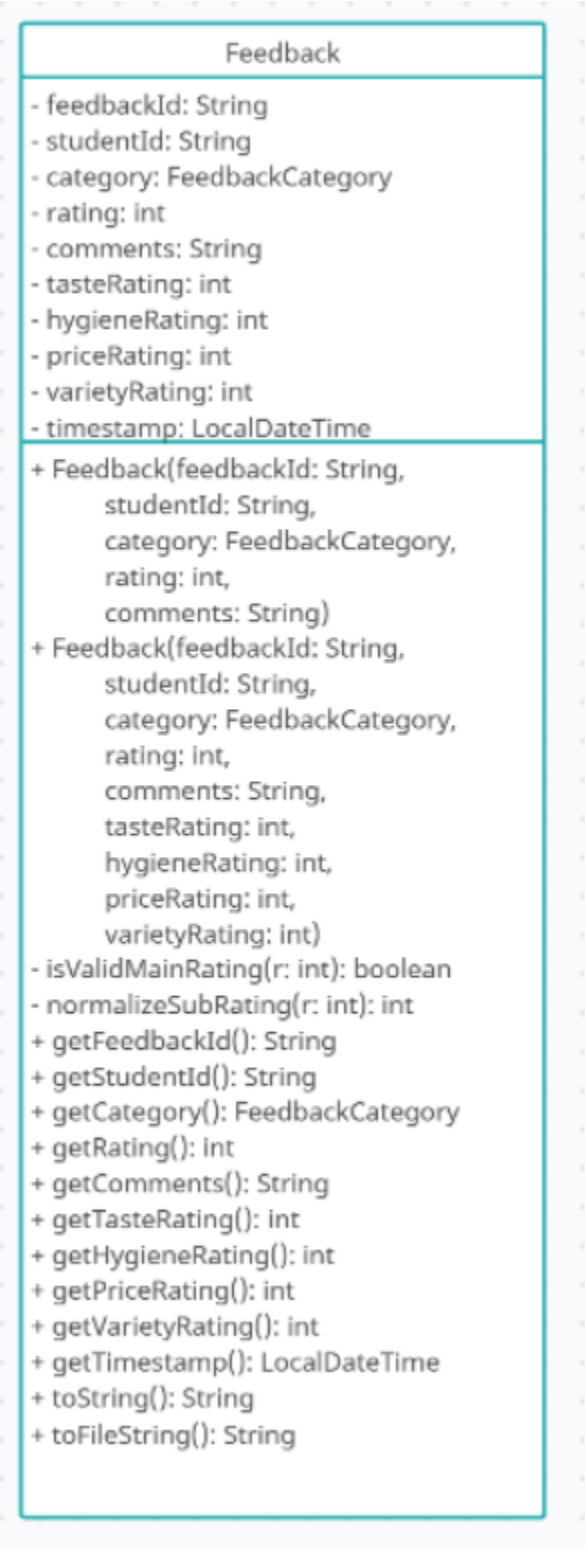

OrderStatus <<enum>>
PENDING
PREPARING
READY
COMPLETED
CANCELLED
CateringStatus <<enum>>
PLANNED
SCHEDULED
COMPLETED
CANCELLED
FeedbackCategory <<enum>>
CANTEEN
MESS
EVENT_CATERING
TransactionType <<enum>>
CREDIT
DEBIT
Payable <<interface>>
+ getAmountToPay(): double
+ getPayerId(): String
Billable <<interface>>
+ calculateBillAmount(): double
+ getBillDescription(): String
Orderable
+ getItemName(): String
+ getUnitPrice(): double
InsufficientBalanceException
extends Exception
+ InsufficientBalanceException(message: String)
InvalidAmountException
extends Exception
+ InvalidAmountException(message: String)
PaymentMethod <<abstract>>
+ pay(payable: Payable) throws InvalidAmountException, InsufficientBalanceException

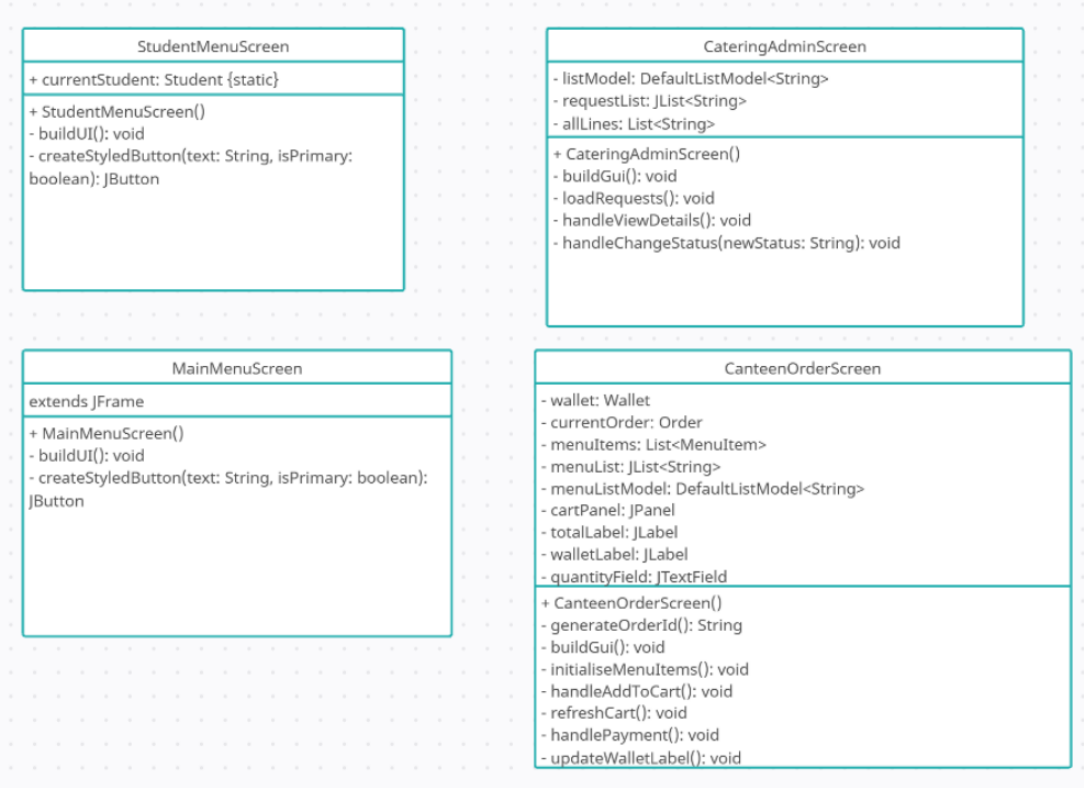

CanteenOrderScreen
- wallet: Wallet
- currentOrder: Order
- menuItems: List
- menuList: JList
- menuListModel: DefaultListModel
- cartPanel: JPanel
- totalLabel: JLabel
- walletLabel: JLabel
- quantityField: JTextField
+ CanteenOrderScreen()
- generateOrderId(): String
- buildGui(): void
- initialiseMenuItems(): void
- handleAddToCart(): void
- refreshCart(): void
- handlePayment(): void
- updateWalletLabel(): void
CateringItem
- menuItem: MenuItem
- quantity: int
+ CateringItem(menuItem: MenuItem, quantity: int)
+ getMenuItem(): MenuItem
+ getQuantity(): int
+ setQuantity(quantity: int): void
+ getSubTotal(): double
+ toString(): String
MessPlanAdminScreen
- subscriptionList: JList
- listModel: DefaultListModel
- detailsArea: JTextArea
- subscriptionData: List
+ MessPlanAdminScreen()
- buildUI(): void
- loadSubscriptions(): void
- showDetails(): void

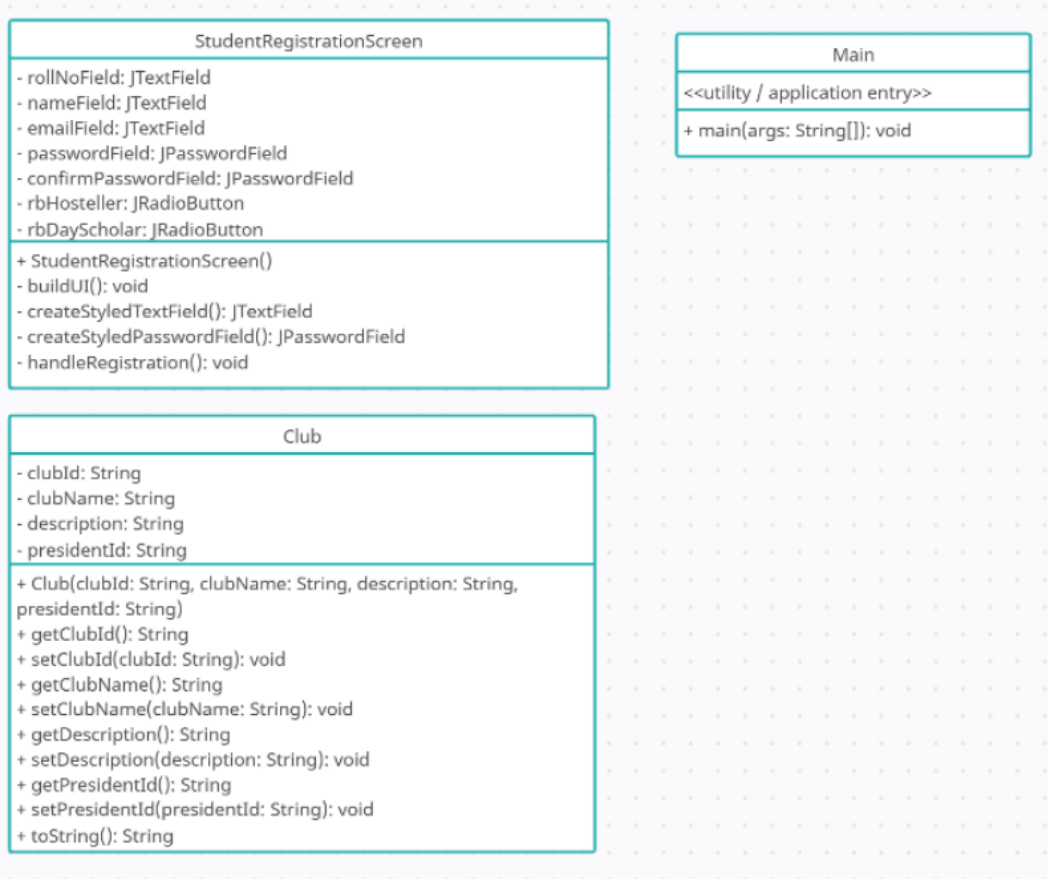

FeedbackAdminScreen
-feedbackList : JList
- listModel : DefaultListModel
- detailsArea : JTextArea
- feedbackData : List
+ FeedbackAdminScreen()
- initComponents() : void
- loadFeedbackFromDatabase() : void
- showSelectedFeedback() : void
FeedbackRecord
- id : String
- studentId : String
- category : String
- rating : int
- comments : String
- timestamp : String

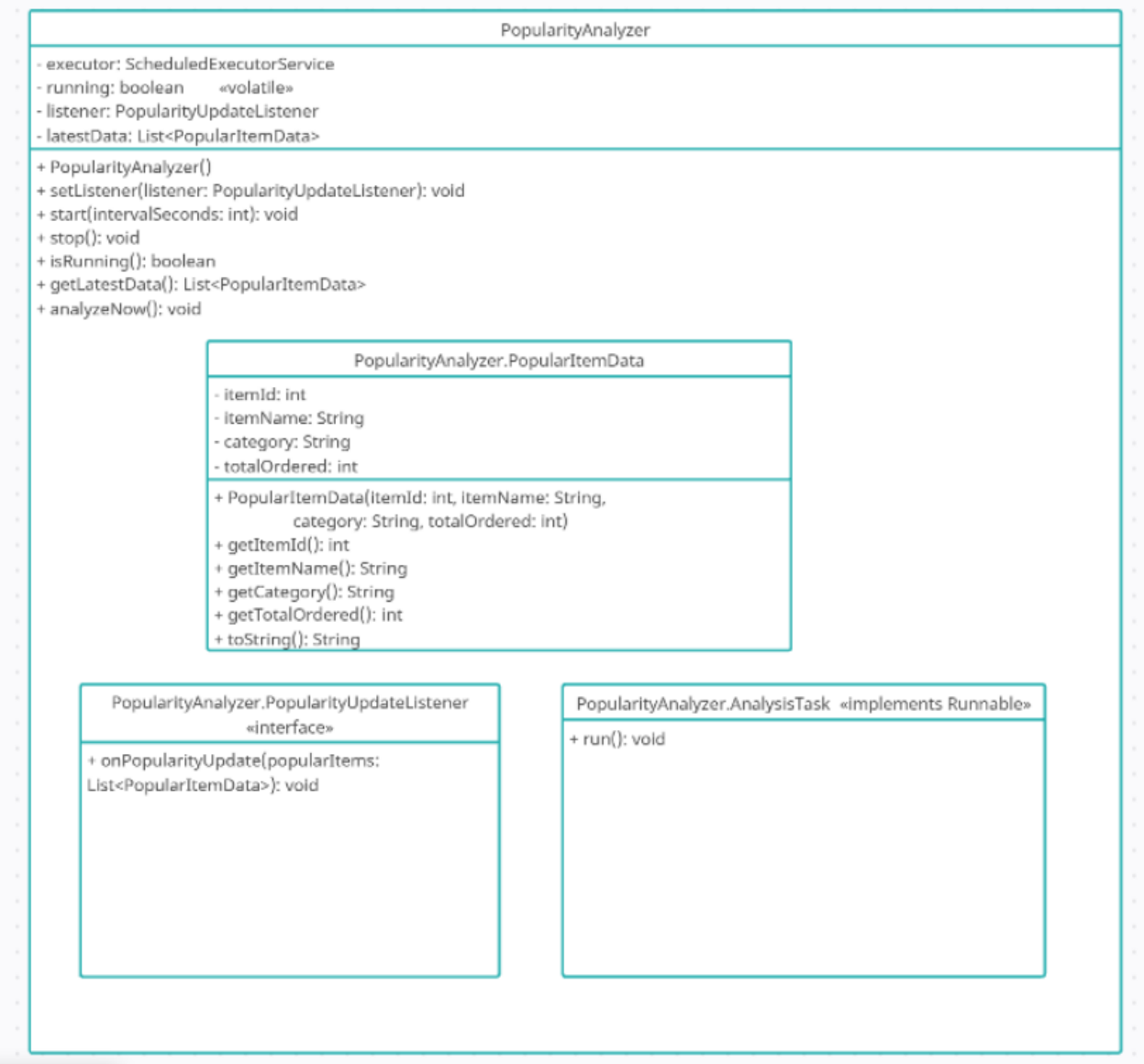

CanteenStaffScreen
- orderList: JList<String>
- listModel: DefaultListModel<String>
- detailsArea: JTextArea
- filterCombo: JComboBox<String>
- summaryLabel: JLabel
- allOrders: List<OrderData>
- filteredOrders: List<OrderData>
+ CanteenStaffScreen()
- initComponents(): void
- loadOrdersFromDatabase(): void
- applyFilterAndRefreshList(): void
- updateSummaryLabel(): void
- showSelectedOrderDetails(): void
- updateOrderStatus(newStatus: String): void
CanteenStaffScreen.OrderData
- id: String
- studentId: String
- total: double
- time: Timestamp
- status: String
- items: List<String>
+ OrderData(id: String, studentId: String,
total: double, time: Timestamp, status: String)

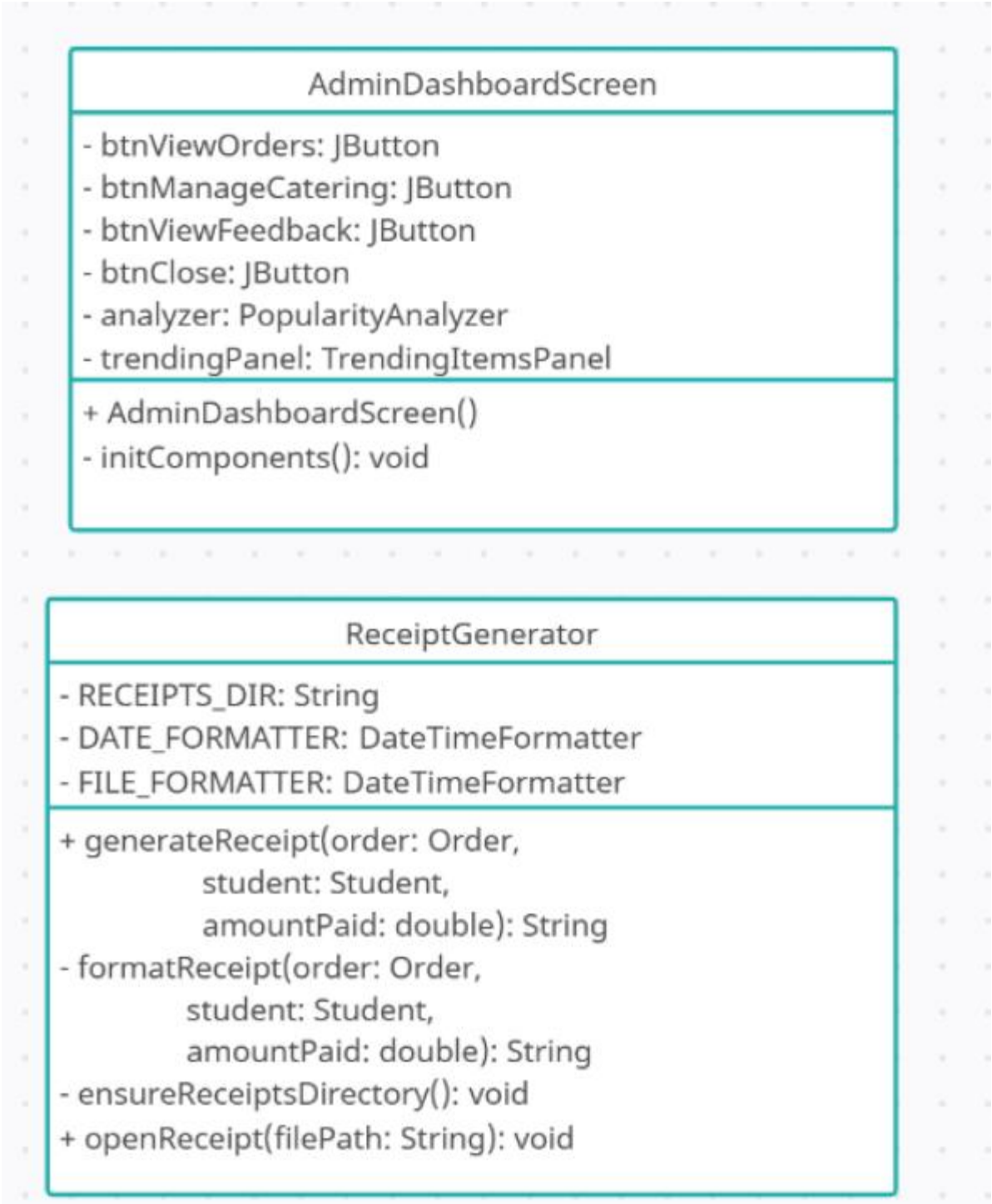

DatabaseManager
- DB_URL: String {static}
- DB_USER: String {static}
- DB_PASSWORD: String {static}
+ getConnection(): Connection
+ authenticateStudent(rollNo: String, password: String): Student
+ authenticateAdmin(username: String, password: String): Admin
- updateLastLogin(studentId: String): void
- updateAdminLastLogin(adminId: String): void
+ registerStudent(student: Student, password: String): boolean
+ subscribeToMessPlan(studentId: String, planId: String): boolean
+ loadStudentMessPlan(studentId: String): MealPlan
+ loadMessPlans(): List<MealPlan>
+ getStudentName(studentId: String): String
+ isHosteller(studentId: String): boolean
+ loadMenuItems(): List<MenuItem>
+ loadWalletBalance(ownerId: String): double
+ saveWalletBalance(wallet: Wallet): void
+ insertTransaction(transaction: Transaction): boolean
+ insertFeedback(feedback: Feedback): void
+ loadStudentFeedback(studentId: String): List<Feedback>
+ loadAllFeedback(): List<Feedback>
+ loadClubs(): List<Club>
+ insertOrder(order: Order): boolean
+ updateOrderStatus(orderId: String, status: OrderStatus): boolean
+ updateMenuItem(item: MenuItem): void
+ deleteMenuItem(itemId: int): void
+ loadOrderItems(orderId: String): List<String>
+ getNextCateringRequestId(): String
+ insertCateringRequest(requestId: String,
eventName: String,
date: String,
time: String,
venue: String,
attendees: int,
items: String,
status: String): void
+ loadCateringRequests(): List<String>
+ updateCateringStatus(requestId: String, newStatus: String): void
+ getPopularMenuItems(limit: int): List<Object[]>

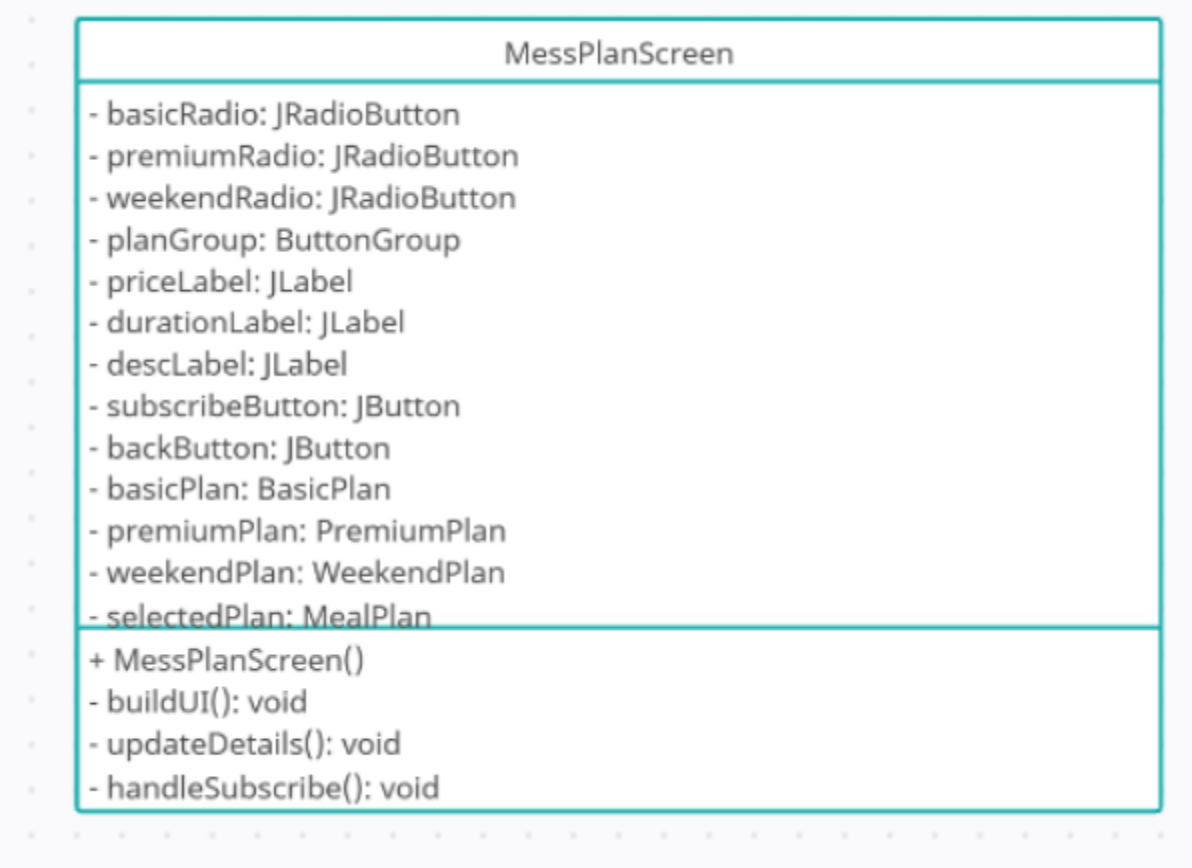

Feedback
- feedbackId: String
- studentId: String
- category: FeedbackCategory
- rating: int
- comments: String
- tasteRating: int
- hygieneRating: int
- priceRating: int
- varietyRating: int
- timestamp: LocalDateTime
+ Feedback(feedbackId: String,
studentId: String,
category: FeedbackCategory,
rating: int,
comments: String)
+ Feedback(feedbackId: String,
studentId: String,
category: FeedbackCategory,
rating: int,
comments: String,
tasteRating: int,
hygieneRating: int,
priceRating: int,
varietyRating: int)
- isValidMainRating(r: int): boolean
- normalizeSubRating(r: int): int
+ getFeedbackId(): String
+ getStudentId(): String
+ getCategory(): FeedbackCategory
+ getRating(): int
+ getComments(): String
+ getTasteRating(): int
+ getHygieneRating(): int
+ getPriceRating(): int
+ getVarietyRating(): int
+ getTimestamp(): LocalDateTime
+ toString(): String
+ toFileString(): String

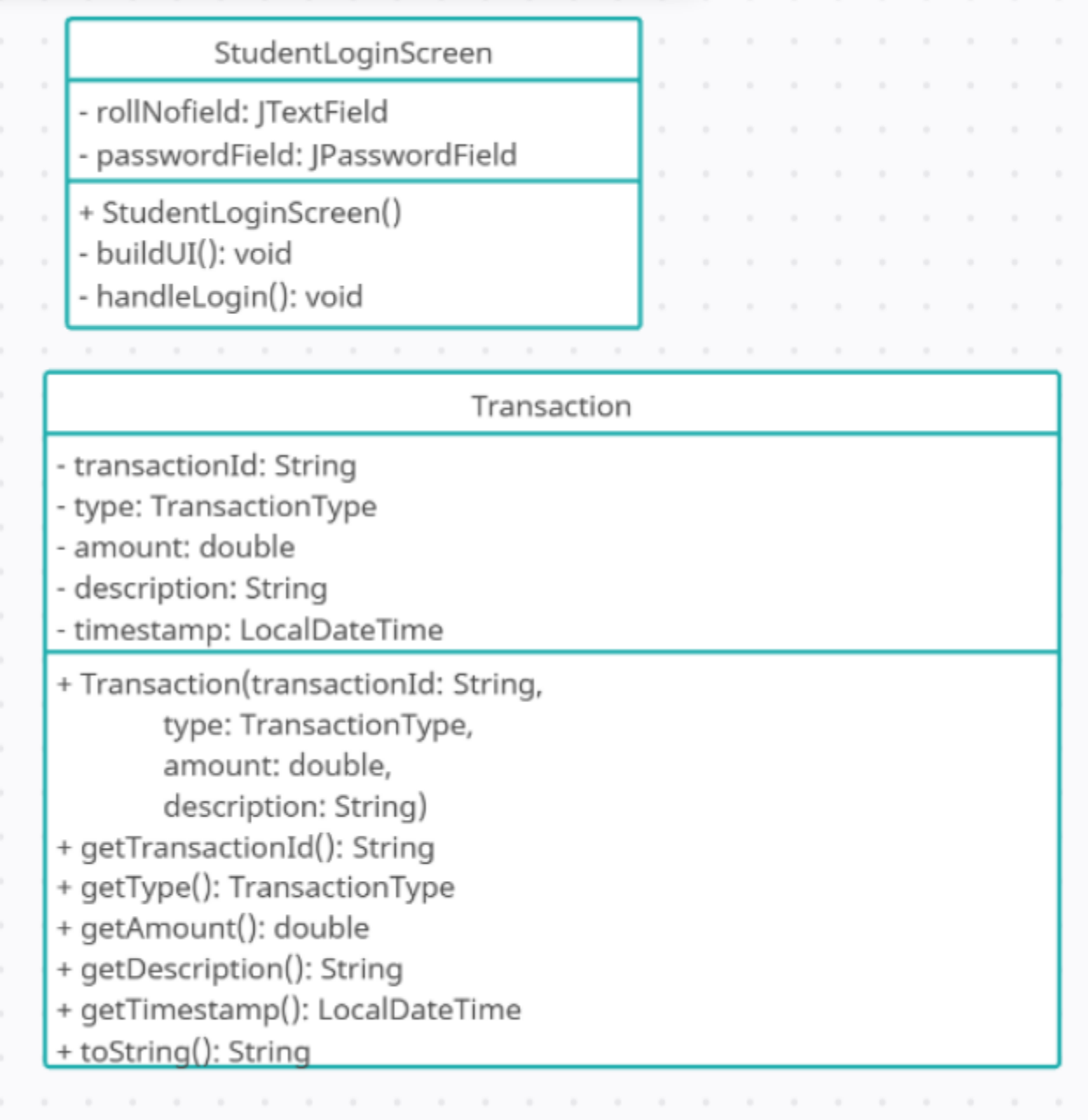

StudentMenuScreen
+ currentStudent: Student {static}
+ StudentMenuScreen()
- buildUI(): void
- createStyledButton(text: String, isPrimary: boolean): JButton
CateringAdminScreen
- listModel: DefaultListModel<String>
- requestList: JList<String>
- allLines: List<String>
+ CateringAdminScreen()
- buildGui(): void
- loadRequests(): void
- handleViewDetails(): void
- handleChangeStatus(newStatus: String): void
MainMenuScreen
extends JFrame
+ MainMenuScreen()
- buildUI(): void
- createStyledButton(text: String, isPrimary: boolean): JButton
CanteenOrderScreen
- wallet: Wallet
- currentOrder: Order
- menuItems: List<MenuItem>
- menuList: JList<String>
- menuListModel: DefaultListModel<String>
- cartPanel: JPanel
- totalLabel: JLabel
- walletLabel: JLabel
- quantityField: JTextField
+ CanteenOrderScreen()
- generateOrderId(): String
- buildGui(): void
- initialiseMenuItems(): void
- handleAddToCart(): void
- refreshCart(): void
- handlePayment(): void
- updateWalletLabel(): void

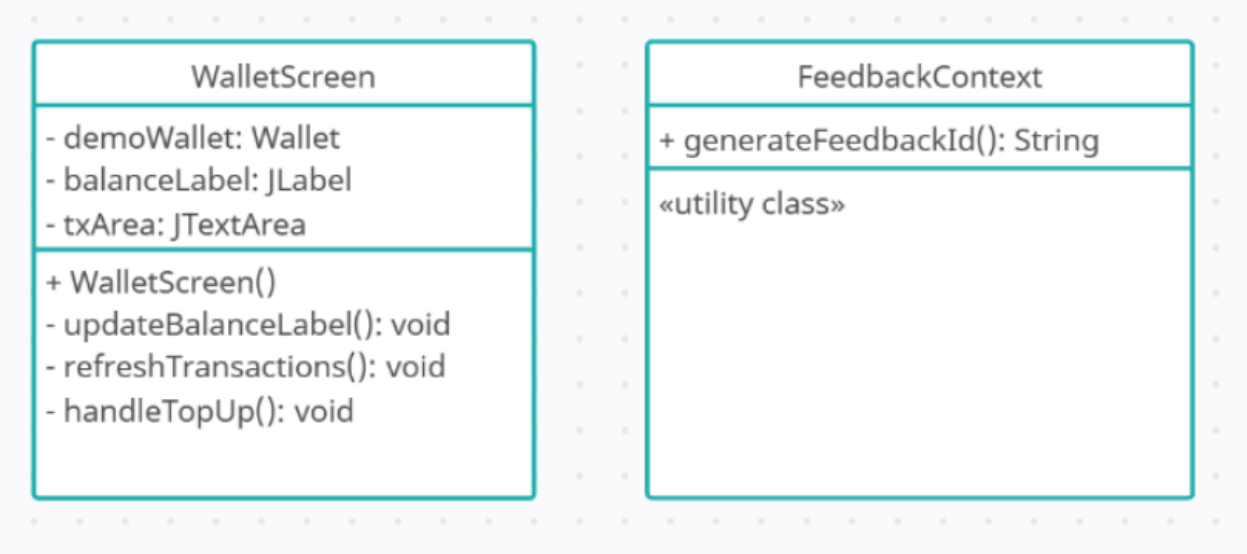

StudentRegistrationScreen
- rollNoField: JTextField
- nameField: JTextField
- emailField: JTextField
- passwordField: JPasswordField
- confirmPasswordField: JPasswordField
- rbHosteller: JRadioButton
- rbDayScholar: JRadioButton
+ StudentRegistrationScreen()
- buildUI(): void
- createStyledTextField(): JTextField
- createStyledPasswordField(): JPasswordField
- handleRegistration(): void
Main
<<utility / application entry>>
+ main(args: String[]): void
Club
- clubId: String
- clubName: String
- description: String
- presidentId: String
+ Club(clubId: String, clubName: String, description: String, presidentId: String)
+ getClubId(): String
+ setClubId(clubId: String): void
+ getClubName(): String
+ setClubName(clubName: String): void
+ getDescription(): String
+ setDescription(description: String): void
+ getPresidentId(): String
+ setPresidentId(presidentId: String): void
+ toString(): String

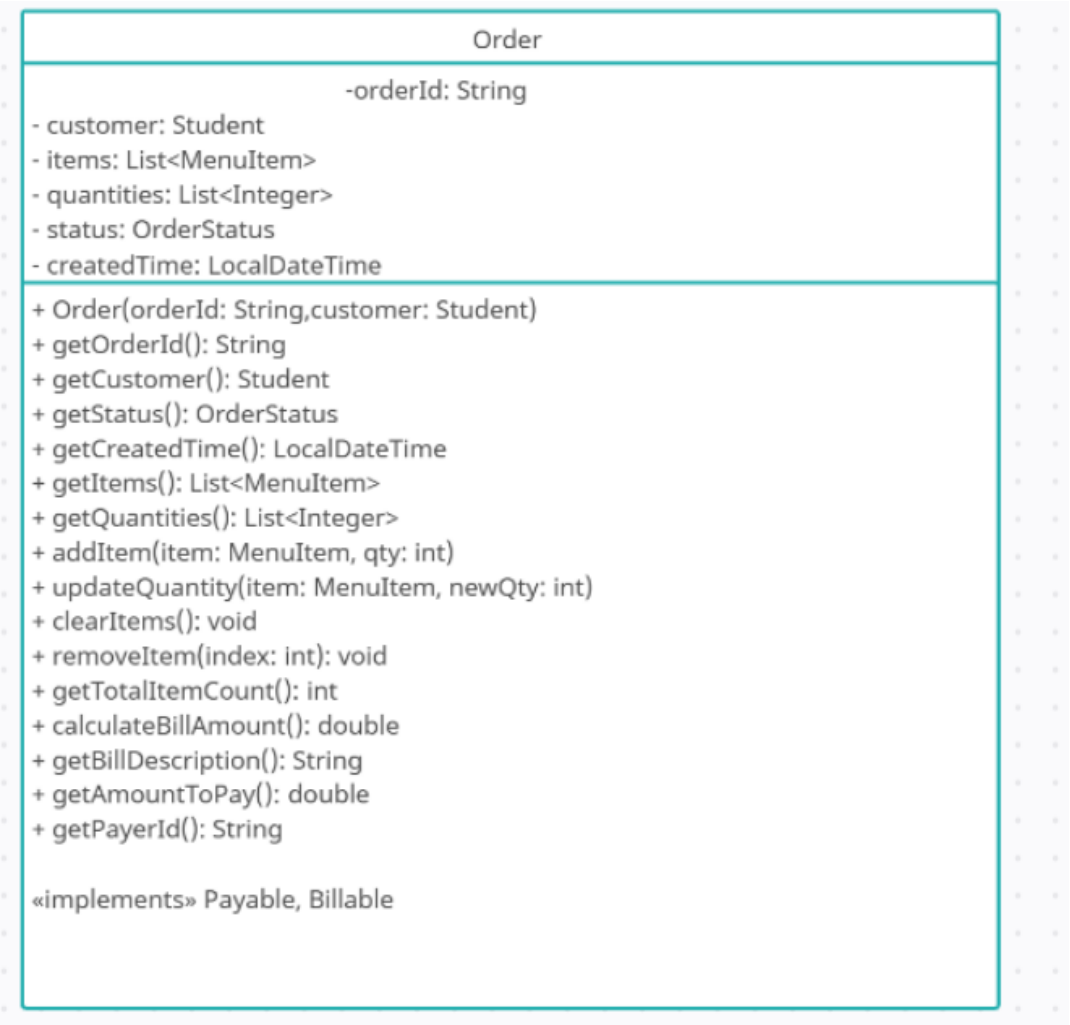

PopularityAnalyzer
- executor: ScheduledExecutorService
- running: boolean «volatile»
- listener: PopularityUpdateListener
- latestData: List<PopularItemData>
+ PopularityAnalyzer()
+ setListener(listener: PopularityUpdateListener): void
+ start(intervalSeconds: int): void
+ stop(): void
+ isRunning(): boolean
+ getLatestData(): List<PopularItemData>
+ analyzeNow(): void
PopularityAnalyzer.PopularItemData
- itemId: int
- itemName: String
- category: String
- totalOrdered: int
+ PopularItemData(itemId: int, itemName: String, category: String, totalOrdered: int)
+ getItemId(): int
+ getItemName(): String
+ getCategory(): String
+ getTotalOrdered(): int
+ toString(): String
PopularityAnalyzer.PopularityUpdateListener «interface»
+ onPopularityUpdate(popularItems: List<PopularItemData>): void
PopularityAnalyzer.AnalysisTask «implements Runnable»
+ run(): void

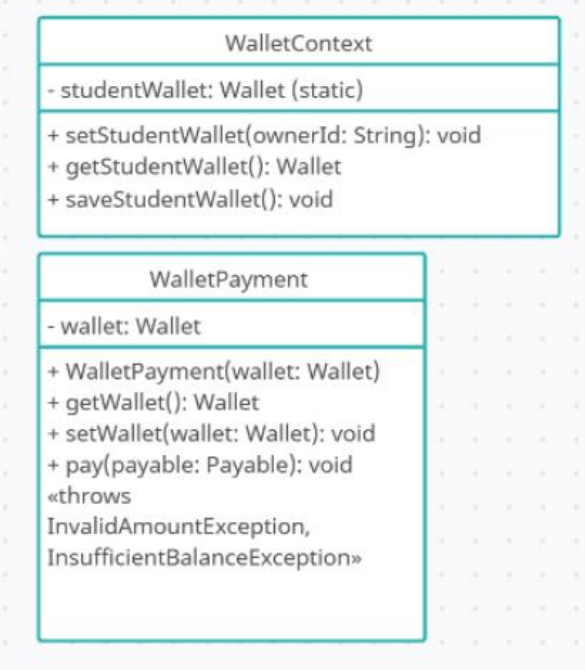

AdminDashboardScreen
- btnViewOrders: JButton
- btnManageCatering: JButton
- btnViewFeedback: JButton
- btnClose: JButton
- analyzer: PopularityAnalyzer
- trendingPanel: TrendingItemsPanel
+ AdminDashboardScreen()
- initComponents(): void
ReceiptGenerator
- RECEIPTS_DIR: String
- DATE_FORMATTER: DateTimeFormatter
- FILE_FORMATTER: DateTimeFormatter
+ generateReceipt(order: Order,
student: Student,
amountPaid: double): String
- formatReceipt(order: Order,
student: Student,
amountPaid: double): String
- ensureReceiptsDirectory(): void
+ openReceipt(filePath: String): void

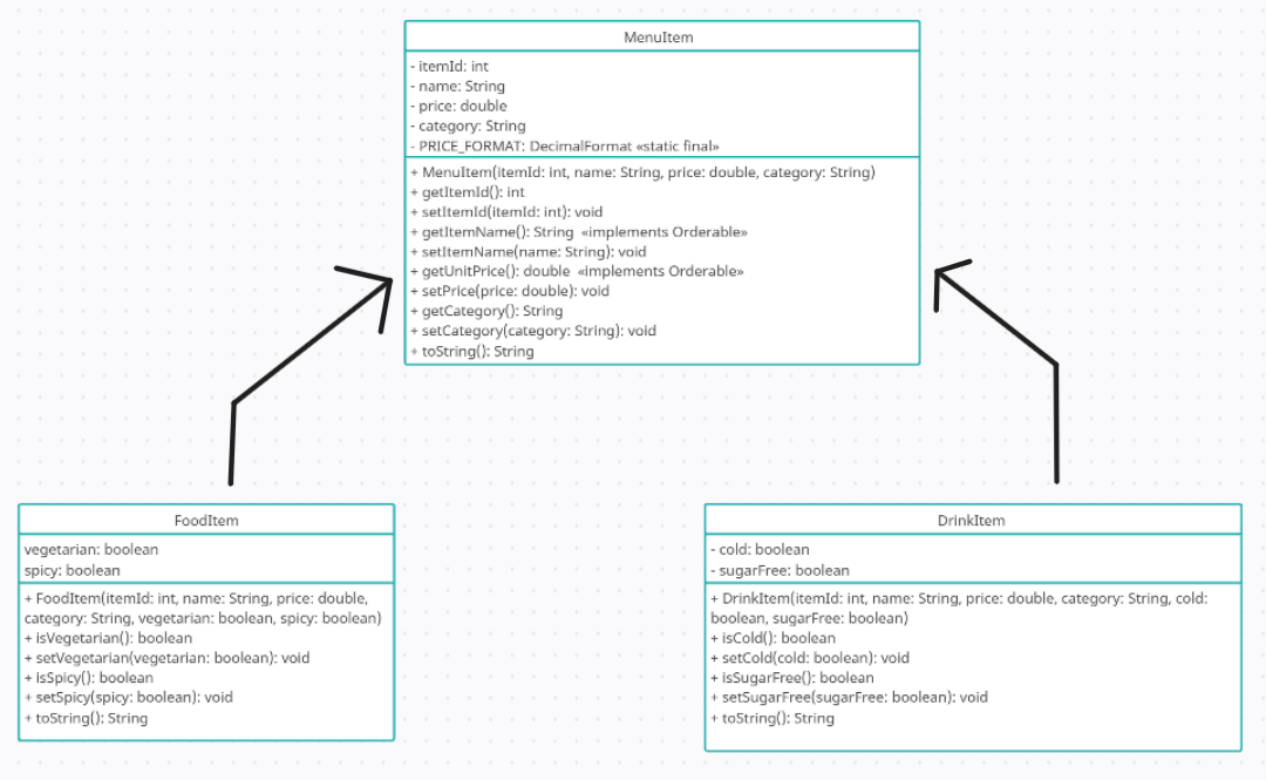

MessPlanScreen
- basicRadio: JRadioButton
- premiumRadio: JRadioButton
- weekendRadio: JRadioButton
- planGroup: ButtonGroup
- priceLabel: JLabel
- durationLabel: JLabel
- descLabel: JLabel
- subscribeButton: JButton
- backButton: JButton
- basicPlan: BasicPlan
- premiumPlan: PremiumPlan
- weekendPlan: WeekendPlan
- selectedPlan: MealPlan
+ MessPlanScreen()
- buildUI(): void
- updateDetails(): void
- handleSubscribe(): void

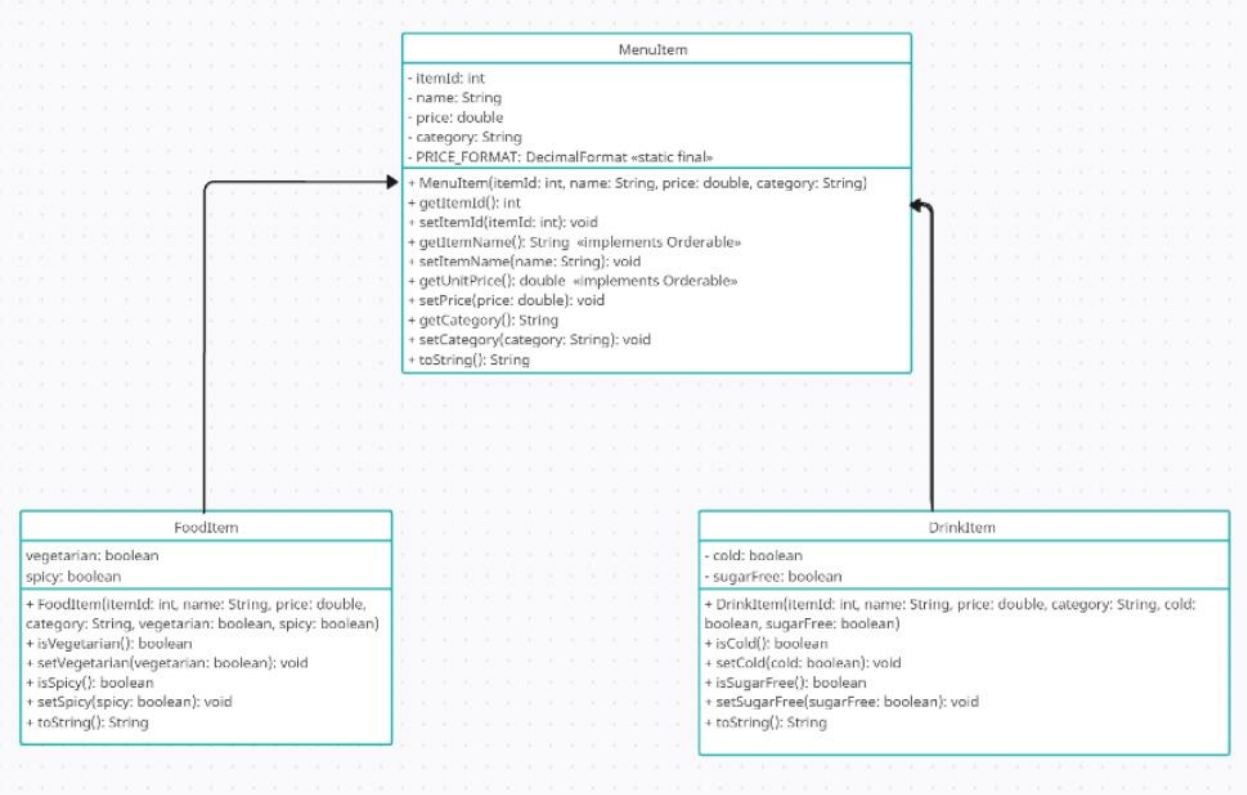

StudentLoginScreen
- rollNofield: JTextField
- passwordField: JPasswordField
+ StudentLoginScreen()
- buildUI(): void
- handleLogin(): void
Transaction
- transactionId: String
- type: TransactionType
- amount: double
- description: String
- timestamp: LocalDateTime
+ Transaction(transactionId: String,
type: TransactionType,
amount: double,
description: String)
+ getTransactionId(): String
+ getType(): TransactionType
+ getAmount(): double
+ getDescription(): String
+ getTimestamp(): LocalDateTime
+ toString(): String

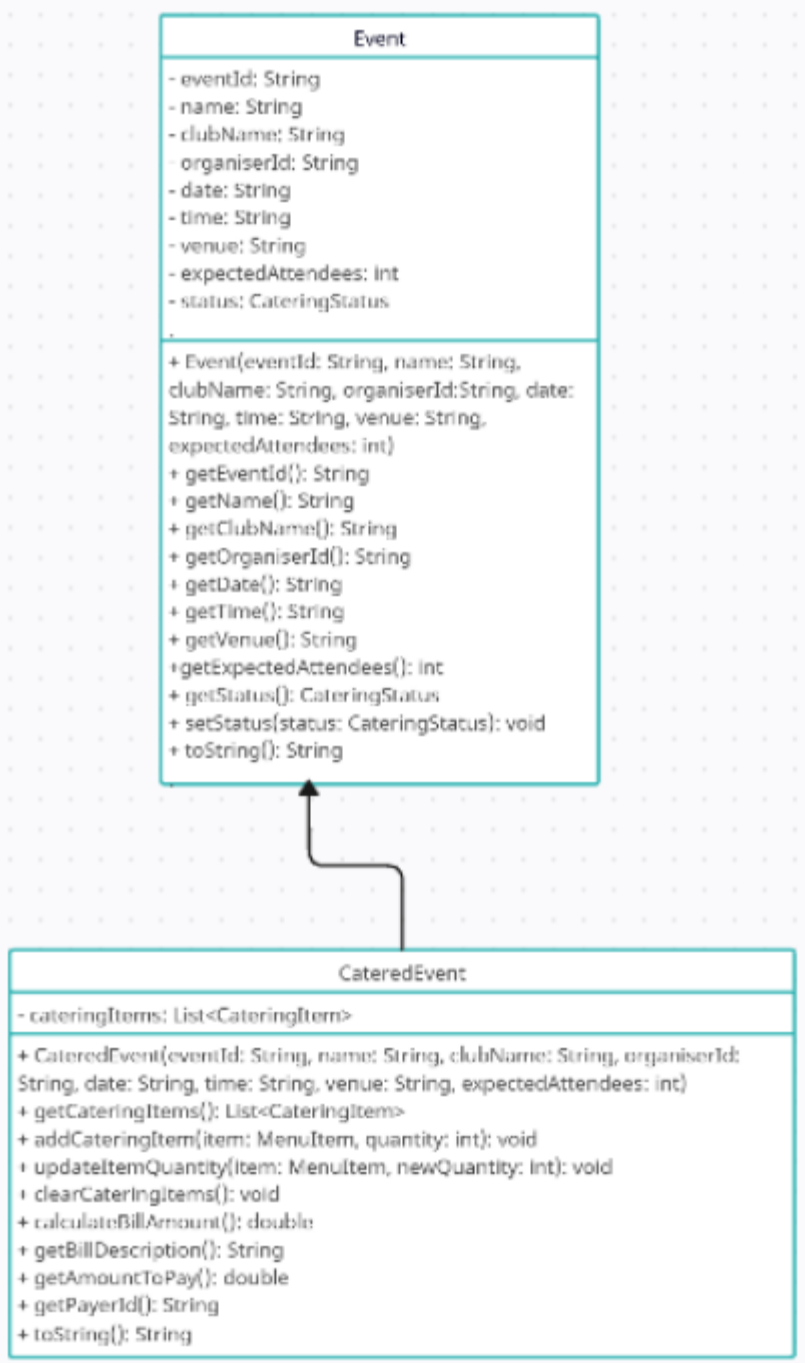

WalletScreen
- demoWallet: Wallet
- balanceLabel: JLabel
- txArea: JTextArea
+ WalletScreen()
- updateBalanceLabel(): void
- refreshTransactions(): void
- handleTopUp(): void
FeedbackContext
+ generateFeedbackId(): String
«utility class»

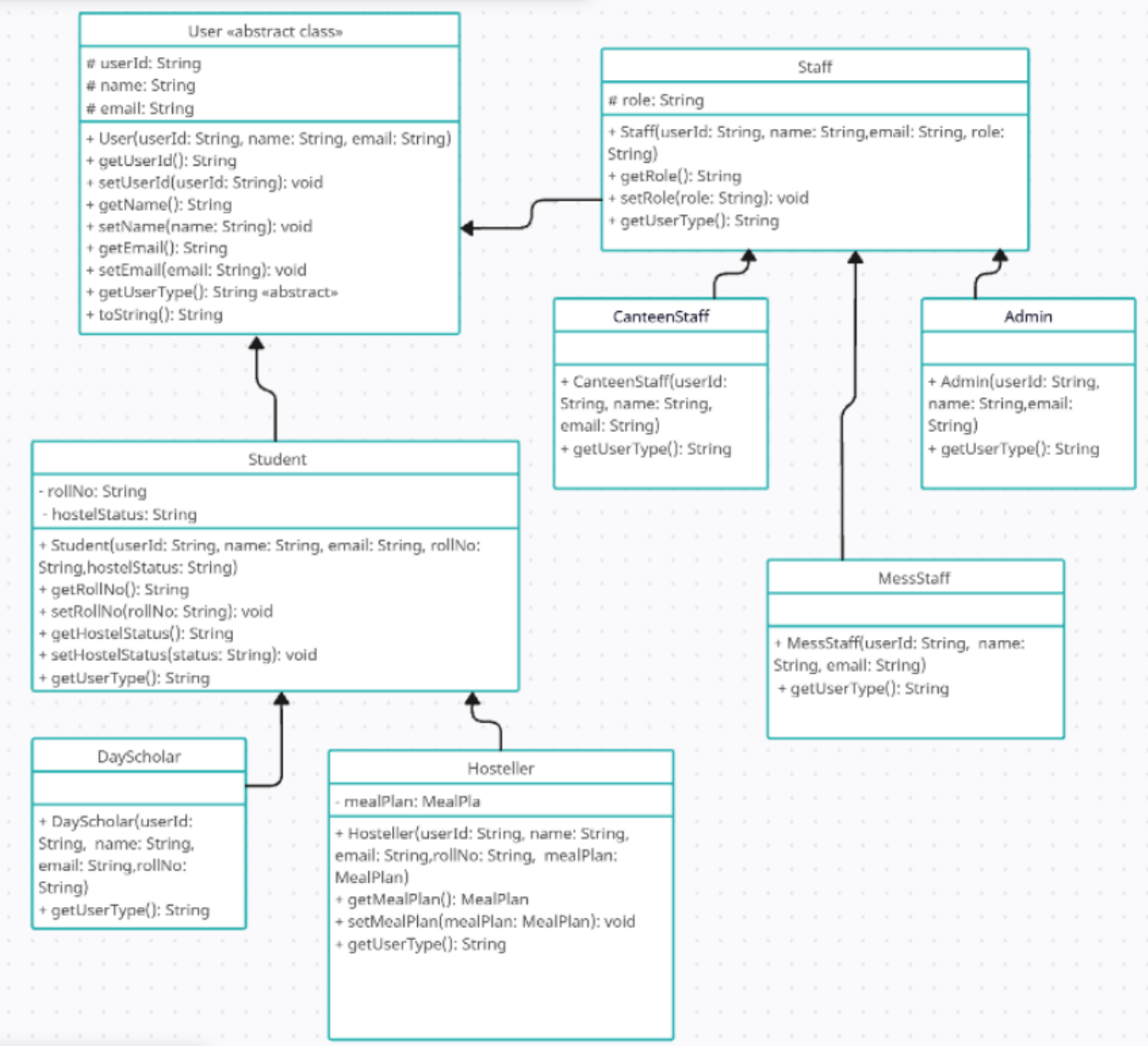

Order
-orderId: String
- customer: Student
- items: List<MenuItem>
- quantities: List<Integer>
- status: OrderStatus
- createdTime: LocalDateTime
+ Order(orderId: String,customer: Student)
+ getOrderId(): String
+ getCustomer(): Student
+ getStatus(): OrderStatus
+ getCreatedTime(): LocalDateTime
+ getItems(): List<MenuItem>
+ getQuantities(): List<Integer>
+ addItem(item: MenuItem, qty: int)
+ updateQuantity(item: MenuItem, newQty: int)
+ clearItems(): void
+ removeItem(index: int): void
+ getTotalItemCount(): int
+ calculateBillAmount(): double
+ getBillDescription(): String
+ getAmountToPay(): double
+ getPayerId(): String
«implements» Payable, Billable

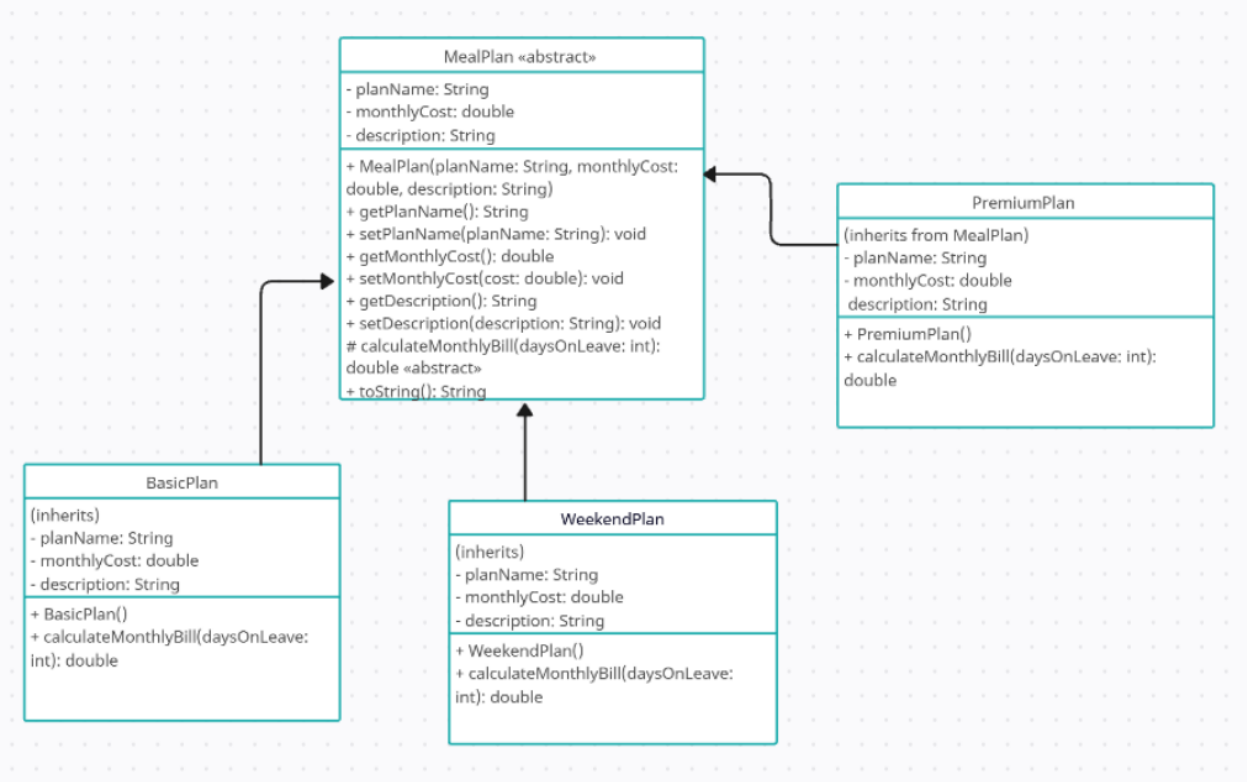

WalletContext
- studentWallet: Wallet (static)
+ setStudentWallet(ownerId: String): void
+ getStudentWallet(): Wallet
+ saveStudentWallet(): void
WalletPayment
- wallet: Wallet
+ WalletPayment(wallet: Wallet)
+ getWallet(): Wallet
+ setWallet(wallet: Wallet): void
+ pay(payable: Payable): void
«throws
InvalidAmountException,
InsufficientBalanceException»


MenuItem
- itemId: int
- name: String
- price: double
- category: String
- PRICE_FORMAT: DecimalFormat «static final»
+ MenuItem(itemId: int, name: String, price: double, category: String)
+ getItemId(): int
+ setItemId(itemId: int): void
+ getItemName(): String «implements Orderable»
+ setItemName(name: String): void
+ getUnitPrice(): double «implements Orderable»
+ setPrice(price: double): void
+ getCategory(): String
+ setCategory(category: String): void
+ toString(): String
FoodItem
vegetarian: boolean
spicy: boolean
+ FoodItem(itemId: int, name: String, price: double, category: String, vegetarian: boolean, spicy: boolean)
+ isVegetarian(): boolean
+ setVegetarian(vegetarian: boolean): void
+ isSpicy(): boolean
+ setSpicy(spicy: boolean): void
+ toString(): String
DrinkItem
- cold: boolean
- sugarFree: boolean
+ DrinkItem(itemId: int, name: String, price: double, category: String, cold: boolean, sugarFree: boolean)
+ isCold(): boolean
+ setCold(cold: boolean): void
+ isSugarFree(): boolean
+ setSugarFree(sugarFree: boolean): void
+ toString(): String


MenuItem
- itemId: int
- name: String
- price: double
- category: String
- PRICE_FORMAT: DecimalFormat «static final»
+ MenuItem(itemId: int, name: String, price: double, category: String)
+ getItemId(): int
+ setItemId(itemId: int): void
+ getItemName(): String «implements Orderable»
+ setItemName(name: String): void
+ getUnitPrice(): double «implements Orderable»
+ setPrice(price: double): void
+ getCategory(): String
+ setCategory(category: String): void
+ toString(): String
FoodItem
vegetarian: boolean
spicy: boolean
+ FoodItem(itemId: int, name: String, price: double, category: String, vegetarian: boolean, spicy: boolean)
+ isVegetarian(): boolean
+ setVegetarian(vegetarian: boolean): void
+ isSpicy(): boolean
+ setSpicy(spicy: boolean): void
+ toString(): String
DrinkItem
- cold: boolean
- sugarFree: boolean
+ DrinkItem(itemId: int, name: String, price: double, category: String, cold: boolean, sugarFree: boolean)
+ isCold(): boolean
+ setCold(cold: boolean): void
+ isSugarFree(): boolean
+ setSugarFree(sugarFree: boolean): void
+ toString(): String


Event
- eventId: String
- name: String
- clubName: String
organiserId: String
- date: String
- time: String
- venue: String
- expectedAttendees: int
- status: CateringStatus
+ Event(eventId: String, name: String, clubName: String, organiserId:String, date: String, time: String, venue: String, expectedAttendees: int)
+ getEventId(): String
+ getName(): String
+ getClubName(): String
+ getOrganiserId(): String
+ getDate(): String
+ getTime(): String
+ getVenue(): String
+getExpectedAttendees(): int
+ getStatus(): CateringStatus
+ setStatus(status: CateringStatus): void
+ toString(): String
CateredEvent
- cateringItems: List<CateringItem>
+ CateredEvent(eventId: String, name: String, clubName: String, organiserId: String, date: String, time: String, venue: String, expectedAttendees: int)
+ getCateringItems(): List<CateringItem>
+ addCateringItem(item: MenuItem, quantity: int): void
+ updateItemQuantity(item: MenuItem, newQuantity: int): void
+ clearCateringItems(): void
+ calculateBillAmount(): double
+ getBillDescription(): String
+ getAmountToPay(): double
+ getPayerId(): String
+ toString(): String


User «abstract class»
# userId: String
# name: String
# email: String
+ User(userId: String, name: String, email: String)
+ getUserId(): String
+ setUserId(userId: String): void
+ getName(): String
+ setName(name: String): void
+ getEmail(): String
+ setEmail(email: String): void
+ getUserType(): String «abstract»
+ toString(): String
Staff
# role: String
+ Staff(userId: String, name: String,email: String, role: String)
+ getRole(): String
+ setRole(role: String): void
+ getUserType(): String
CanteenStaff
+ CanteenStaff(userId: String, name: String, email: String)
+ getUserType(): String
Admin
+ Admin(userId: String, name: String,email: String)
+ getUserType(): String
Student
- rollNo: String
- hostelStatus: String
+ Student(userId: String, name: String, email: String, rollNo: String,hostelStatus: String)
+ getRollNo(): String
+ setRollNo(rollNo: String): void
+ getHostelStatus(): String
+ setHostelStatus(status: String): void
+ getUserType(): String
MessStaff
+ MessStaff(userId: String, name: String, email: String)
+ getUserType(): String
DayScholar
+ DayScholar(userId: String, name: String, email: String,rollNo: String)
+ getUserType(): String
Hosteller
- mealPlan: MealPla
+ Hosteller(userId: String, name: String, email: String,rollNo: String, mealPlan: MealPlan)
+ getMealPlan(): MealPlan
+ setMealPlan(mealPlan: MealPlan): void
+ getUserType(): String


MealPlan «abstract»
- planName: String
- monthlyCost: double
- description: String
+ MealPlan(planName: String, monthlyCost: double, description: String)
+ getPlanName(): String
+ setPlanName(planName: String): void
+ getMonthlyCost(): double
+ setMonthlyCost(cost: double): void
+ getDescription(): String
+ setDescription(description: String): void
# calculateMonthlyBill(daysOnLeave: int): double «abstract»
+ toString(): String
PremiumPlan
(inherits from MealPlan)
- planName: String
- monthlyCost: double
description: String
+ PremiumPlan()
+ calculateMonthlyBill(daysOnLeave: int): double
BasicPlan
(inherits)
- planName: String
- monthlyCost: double
- description: String
+ BasicPlan()
+ calculateMonthlyBill(daysOnLeave: int): double
WeekendPlan
(inherits)
- planName: String
- monthlyCost: double
- description: String
+ WeekendPlan()
+ calculateMonthlyBill(daysOnLeave: int): double


CateringRequestScreen
- eventNameField : JTextField
- dateField : JTextField
- timeField : JTextField
- venueField : JTextField
- attendeesField : JTextField
- itemCombo : JComboBox<String>
- quantityField : JTextField
- itemsArea : JTextArea
- itemLines : List<String>
+ CateringRequestScreen()
- buildGui() : void
- handleAddItem() : void
- refreshItemsArea() : void
- handleSubmit() : void

# Major Modules

**Student Dashboard System**

- **Canteen Ordering Module**
  - Displays the digital menu with food and drink categories.
  - Allow students to add items to a shopping cart and adjust quantities.
  - Calculates total cost and processes secure payments via the digital wallet.
  - Saves confirmed orders to the database for kitchen processing.
- **Wallet & Transaction Module**
  - Shows the student's current available balance in real-time.
  - Maintains a detailed history of all credits (recharges) and debits (purchases).
  - Provides functionality to simulate adding funds to the wallet.
- **Feedback Module**
  - Provides a structured form for students to rate services on a 1-5 scale.
  - Collects specific ratings for taste, hygiene, price, and variety.
  - Stores comments and timestamps to help administration improve services.
- **Event Catering Module**
  - Enables student clubs to book catering services for campus events.
  - Captures event details including date, time, venue, and expected attendance.
  - Allows selection of specific menu items for the event catering package.
- **Mess Plan Subscription Module**
  - Displays details of available meal plans: Basic, Premium, and Weekend.
  - Allow hostellers to subscribe to a new plan or switch their existing one.
  - Calculates monthly billing based on the chosen plan type.

**Admin Dashboard System**

- **Order Management Module**
  - Provides a live view of the queue for all incoming canteen orders.
  - Allows kitchen staff to update order status (e.g., from "Pending" to "Ready").
  - Helps manage kitchen workflow and notify students when food is prepared.
- **Catering Management Module**
  - Lists all catering requests submitted by student clubs.
  - Allows administrators to review event details and approve or reject requests.
  - Tracks the status of scheduled catering events (Planned vs. Completed).
- **Feedback Review Module**
  - Aggregates all feedback submissions from the student body.
  - Displays ratings and comments categorized by Canteen, Mess, or Events.
  - Helps administrators identify trends and areas needing operational improvement.
- **Mess Plan Administration Module**

- Lists all students currently subscribed to a mess meal plan.
- Shows the specific plan type (Basic/Premium/Weekend) for each student.
- Assists in resource planning and monthly billing verification.

## Implementation Analysis

OOP Concepts Implementation

1. Inheritance

- Where it is implemented: In the User Hierarchy.
- Description: The application uses a base class to hold common data (Name, Email, ID) while specific classes extend this to add specialized roles.
- Files Involved: User.java (Parent), Student.java (Child), Admin.java (Child).

Implementation Code Structure:

```
1
2   public class Admin extends Staff
3   {
4       public Admin(String userId, String name, String email) {
5           super(userId, name, email, "Admin");
6       }
7
8       @Override
9       public String getUserType() {
10          return "Admin";
11      }
12  }
```

2. Abstraction

- Where it is implemented: In the User class and MealPlan definitions.
- Description: The User class is declared as abstract because a generic "User" should never exist in the system; it must always be a specific type (Student/Admin). Similarly, MealPlan hides the calculation logic.
- Files Involved: User.java, MealPlan.java.

Implementation Code Structure:

```java

public abstract class User {
    protected String userId;
    protected String name;
    protected String email;

    public User(String userId, String name, String email) {
        this.userId = userId;
        this.name = name;
        this.email = email;
    }

    public String getUserId() {
        return userId;
    }

    public void setUserId(String userId) {
        this.userId = userId;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    // Subclasses define user type
    public abstract String getUserType();
```

3. Interfaces

- Where it is implemented: In the Financial & Payment Module.

- Description: The Payable interface defines a contract for any entity that requires payment processing. It enforces the implementation of methods to retrieve the transaction amount and the identity of the payer.
- Files Involved: AppConstants.java

Implementation Code Structure:

```
interface Payable {
    double getAmountToPay();

    String getPayerId();
}

interface Billable {
    double calculateBillAmount();

    String getBillDescription();
}

interface Orderable {
    String getItemName();

    double getUnitPrice();
}
```

4. Polymorphism

- Where it is implemented: In Hosteller Subscriptions (Strategy Pattern).
- Description: The system treats different objects (BasicPlan, PremiumPlan) as the same type (MealPlan) at runtime. This allows the Hosteller to switch plans dynamically without breaking the code.
- Files Involved: Hosteller.java.

Implementation Code Structure:

```java

public class Hosteller extends Student {
    private MealPlan mealPlan;

    public Hosteller(String userId, String name, String email, String rollNo, MealPlan mealPlan) {
        super(userId, name, email, rollNo, "Hosteller");
        this.mealPlan = mealPlan;
    }

    public MealPlan getMealPlan() {
        return mealPlan;
    }

    public void setMealPlan(MealPlan mealPlan) {
        this.mealPlan = mealPlan;
    }

    @Override
    public String getUserType() {
        return "Hosteller Student";
    }
}
```

5. Encapsulation

- Where it is implemented: In the Wallet System.
- Description: Critical data like the wallet balance is hidden from direct access. It can only be modified through controlled methods to prevent invalid states (e.g., negative balance).
- Files Involved: Wallet.java.

Implementation Code Structure:

```java


import java.util.ArrayList;
import java.util.List;

public class Wallet {
    private String ownerId; // studentId or clubId
    private double balance;
    private List<Transaction> transactions;

    public Wallet(String ownerId) {
        this.ownerId = ownerId;
        this.balance = 0.0;
        this.transactions = new ArrayList<Transaction>();
    }

    public String getOwnerId() {
        return ownerId;
    }

    public double getBalance() {
        return balance;
    }

    public List<Transaction> getTransactions() {
        return new ArrayList<Transaction>(transactions);
    }

    public void credit(double amount, String description)
            throws InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException("Credit amount must be positive.");
        }
        balance += amount;
        Transaction t = new Transaction(generateTransactionId(),
                TransactionType.CREDIT, amount, description);
        transactions.add(t);
    }

    public void debit(double amount, String description)
            throws InvalidAmountException, InsufficientBalanceException {
        if (amount <= 0) {
            throw new InvalidAmountException("Debit amount must be positive.");
        }
        if (amount > balance) {
            throw new InsufficientBalanceException(
                    "Not enough balance in wallet. Required: " + amount +
                            ", Available: " + balance);
```

## Advanced Java Features

1. JDBC Database Connectivity

- Where it is implemented: In the DatabaseManager class.
- Description: This module manages the connection to the MySQL database using the MySQL Connector JAR included in the project.
- Files Involved: DatabaseManager.java.

Implementation Code Structure:

```java

public class DatabaseManager {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/campus_cafe";
    private static final String DB_USER = "root";
    private static final String DB_PASSWORD = "Password Here"; // Change this to your MySQL password

    // Load MySQL JDBC driver
    static {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            System.out.println("JDBC driver loaded successfully.");
        } catch (ClassNotFoundException e) {
            System.out.println("MySQL JDBC driver not found.");
        }
    }

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
    }
```

2. Exception Handling

- Where it is implemented: Around all Database Operations.
- Description: To prevent the application from crashing if the database is offline, operations are wrapped in try-catch blocks.
- Files Involved: DatabaseManager.java, LoginScreen.java.

Implementation Code Structure:

```java
    try (Connection conn = getConnection();
         PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setString(1, username);
        stmt.setString(2, password);
        ResultSet rs = stmt.executeQuery();

        if (rs.next()) {
            String adminId = rs.getString("admin_id");
            String name = rs.getString("name");
            String email = rs.getString("email");

            // Update last login
            updateAdminLastLogin(adminId);

            return new Admin(adminId, name, email);
        }
    } catch (SQLException e) {
        System.out.println("Error authenticating admin: " + e.getMessage());
    }
    return null;
}
```

3. GUI Implementation (Java Swing)

- Where it is implemented: In the *Screen.java classes (Presentation Layer).
- Description: Uses JFrame for windows, JPanel for layouts.

Implementation Code Structure:

```
10  public class StudentLoginScreen extends JFrame {
11
12      private JTextField rollNofield;
13      private JPasswordField passwordField;
```

```
15      public StudentLoginScreen() {
16          setTitle("Student Login  Campus Café");
17          setSize(450, 480);
18          setLocationRelativeTo(null);
19          setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
20          setResizable(false);
21
22          buildUI();
23      }
```

4. Multithreading

- Where it is implemented: In the Popularity Analysis module (specifically in PopularityAnalyzer class).
- Description: Multithreading is used to analyze menu item popularity asynchronously. When the system needs to fetch trending items, a background thread queries the database and processes the data before automatically updating the UI, ensuring the main GUI remains responsive.
- Files Involved: PopularityAnalyzer.java, TrendingItemsPanel.java

Implementation Code Structure:

```
private class AnalysisTask implements Runnable {
    @Override
    public void run() {
        try {
            // Query database for popular items
            List<Object[]> rawData = DatabaseManager.getPopularMenuItems(5);

            // Convert to PopularItemData objects
            List<PopularItemData> popularItems = new java.util.ArrayList<>();
            for (Object[] row : rawData) {
                PopularItemData item = new PopularItemData(
                        (Integer) row[0], // item_id
                        (String) row[1], // item_name
                        (String) row[2], // category
                        (Integer) row[3] // total_ordered
                );
                popularItems.add(item);
            }

            // Update latest data (synchronized for thread safety)
            synchronized (PopularityAnalyzer.this) {
                latestData = popularItems;
            }

            // Notify listener on Swing EDT (thread-safe UI update)
            if (listener != null && running) {
                SwingUtilities.invokeLater(() -> {
                    listener.onPopularityUpdate(popularItems);
                });
            }

        } catch (Exception e) {
            System.err.println("Error in popularity analysis: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

# Conclusion & Future Work:

## Conclusion:

The Campus Café – Ordering & Food Management System successfully achieved its objective of creating a unified and user-friendly platform to manage all food-related activities on campus. The system brings together the key functions that include canteen ordering, wallet payments, mess plan subscriptions, event requests for catering, and feedback submission into one accessible application and simplifies daily processes for students and administrators.

The critical success of this project is the clean and organized partitioning of the system by the different user types. The students themselves can register, log in, make orders for the canteen, handle the digital wallet, submit feedback, or raise requests for catering to club events. The subscription module for the Mess Plan is hosted only for hostellers. For the administrators, this involves having full control in managing the system, viewing and updating the order queue to the canteen, processing requests for catering, observing feedback submissions, and monitoring mess plan subscriptions. This is done in a very structured fashion so that the users only interface with the portions meant for them.

The system's digital wallet was integrated, allowing smooth, cashless transactions. Other tasks, which proved time-consuming, like order status updates, catering request handling, and feedback reviews, were automated to make the campus food-service workflow more efficient and reliable. All the core modules were developed, integrated, and extensively tested to ensure consistent performance of the system in various scenarios.

In all, the project delivered a complete and functional prototype that meets all planned requirements, showing very strong application of Object-Oriented Programming concepts, GUI design, and structured system development, resulting in a reliable solution that simplifies food-related operations for both students and administrators.

## Future Work:

1) Mobile Application and Real-Time Notifications

    - Enhancement: Develop an Inventory Management Module that tracks raw material stock levels, consumption per order (based on recipe components), and automatically generates purchase orders when stock falls below defined thresholds.
    - Benefit: This moves the system beyond order fulfillment and into true café management, allowing for cost control, waste reduction, and predictive stocking. This is vital for accurate financial reporting.

2) Expanded Payment Gateway and Wallet Security

- Enhancement: Currently, payment is internal via the wallet. Future work should be integrated with external Payment Gateways (like PayPal) to allow students to top up their wallet directly via Credit Card/UPI, rather than relying on manual processing.
- Benefit: Increases the convenience of the Wallet top-up process and improves the system's overall financial security by offloading payment data handling to certified providers.

3) Comprehensive Reporting and Analytics Module

- Enhancement: Develop a dedicated Reporting Module accessible via the Admin Dashboard capable of generating detailed reports on: Sales Trends, Menu Popularity and Financials
- Benefit: Provides critical Business Intelligence (BI) to management for making informed decisions on menu adjustments, pricing, and resource allocation.

## References


[1] Kashipara, “Coffee Shop Management System in Java,” Kashipara.com, 2025. [Online]. Available: https://www.kashipara.com/project/java/3583/coffee-shop-management-in-java

[2] Code Projects, “Coffee Shop System in Java with Source Code,” *Code-projects.org*, 2025. [Online]. Available: https://code-projects.org/coffee-shop-system-in-java-with-source-code/

[3] J. W. Cooper, *Java™ Design Patterns: A Tutorial*. Boston, MA, USA: Addison-Wesley, 2000.

[4] W3Schools, “Java Tutorial,” *W3Schools.com*, 2025. [Online]. Available: https://www.w3schools.com/java/

[5] Oracle, “Java Documentation and Tutorials,” *dev.java*, 2025. [Online]. Available: https://dev.java/

[6] Codecademy, “Learn Java – Free Interactive Course,” *Codecademy*, 2025. [Online]. Available: https://www.codecademy.com/learn/learn-java

[7] MySQL, “MySQL 8.0 Reference Manual,” *dev.mysql.com*, 2025. [Online]. Available: https://dev.mysql.com/doc/

[8] BlueJ, “BlueJ Official Documentation,” *bluej.org*, 2025. [Online]. Available: https://bluej.org/

[9] GeeksforGeeks, “Java Programming Language Tutorials,” *GeeksforGeeks.org*, 2025. [Online]. Available: https://www.geeksforgeeks.org/java/

[10] Tutorialspoint, “Java Object-Oriented Programming,” *Tutorialspoint.com*, 2025. [Online]. Available: https://www.tutorialspoint.com/java/

[11] Oracle, “JDBC Overview: Java Database Connectivity,” *Oracle.com*, 2025. [Online]. Available: https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/

[12] StackOverflow, “Java Swing and JDBC Discussions,” *StackOverflow.com*, Accessed 2025. [Online]. Available: https://stackoverflow.com/